

PREDICTION AND MONITORING OF VESSEL SPEED UNDER ENVIRONMENTAL INFLUENCES USING MACHINE LEARNING TECHNIQUES

MUHAMMAD HAFIZ HAMZAH BIN FANSURI 2212803



**DEPARTMENT OF MECHATRONICS ENGINEERING
KULLIAH OF ENGINEERING
INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA
JUN 2026**

PREDICTION AND MONITORING OF VESSEL SPEED UNDER ENVIRONMENTAL INFLUENCES USING MACHINE LEARNING TECHNIQUES

MUHAMMAD HAFIZ HAMZAH BIN FANSURI 2212803

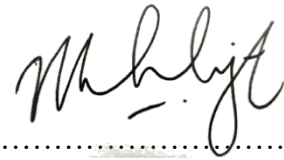
**Project Supervisor: Assoc. Prof. Ir. Dr. NOOR HAZRIN HANY Bt
MOHAMD HANIF**



**A REPORT SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENT FOR A DEGREE OF BACHELOR OF
MECHATRONICS ENGINEERING WITH HONOURS**

APPROVAL

I certify that I have supervised and read this study and that in my opinion, it conforms to acceptable standards of scholarly presentation and is fully adequate, in scope and quality, as Final Year Project report as partial fulfilment for a degree of Bachelor of Engineering (Mechatronics)(Honors).



Assoc. Prof. Ir. Dr. NOOR HAZRIN HANY Bt MOHAMD
HANIF

Supervisor

ABSTRACT

Vessel speed is a critical parameter in maritime operations, directly influencing fuel consumption, schedule adherence, and navigational safety. However, accurate prediction remains challenging due to varying environmental conditions such as wind speed, wave height, and sea currents, which are often not fully captured in conventional models. This study investigates the influence of these environmental factors on vessel speed and develops a hybrid grey-box predictive framework for real-time monitoring and operational support. Operational data provided by a vessel operating company were analyzed to study speed patterns, generate heatmaps, and evaluate correlations between vessel speed and environmental variables, revealing that wind and wave height have a slight negative impact on speed, while vessel operational control remains the dominant factor. Based on these findings, a hybrid model combining a Physics-Informed Neural Network (PINN) with XGBoost was developed and tested using offline vessel datasets, achieving an RMSE of 0.246 knots and an R^2 of 0.9991, demonstrating strong capability in capturing both steady-state behaviour and short-term fluctuations. The model was then deployed on a hardware prototype consisting of an ESP32 microcontroller, an Airmar 200WX weather station, and a BNO085 IMU, and validated through three iterative road tests on the IIUM. The final system achieved a 99.8% of moving-phase predictions within ± 0.5 knots. Overall, the results demonstrate that the proposed hybrid grey-box framework is highly accurate, adaptive, and suitable for real-time vessel speed monitoring and future maritime deployment.

ACKNOWLEDGEMENTS

I would like to express my sincere gratitude to Assoc. Prof. Ir. Dr. Noor Hazrin Hany Bt Mohamad Hanif for her invaluable guidance, constructive advice, and continuous support throughout the duration of this Final Year Project. Her expertise, patience, and encouragement greatly contributed to the successful completion of this work.

I would also like to extend my appreciation to the International Islamic University Malaysia (IIUM) for providing the necessary facilities, resources, and a supportive academic environment that made this project possible.

Finally, I am deeply grateful to my family and friends for their unwavering encouragement, understanding, and moral support throughout this journey, which motivated me to persevere and complete this project successfully.

CONTENTS

APPROVAL.....	1
Abstract	2
Acknowledgements.....	3
List of Tables	7
List of Figures	8
List of Abbreviations	10
CHAPTER 1	11
INTRODUCTION	11
1.1 Background.....	11
1.2 PROBLEM STATEMENT.....	11
1.3 SCOPE OF PORJECT	12
1.4 OBJECTIVES.....	13
CHAPTER 2	14
LITERATURE REVIEW	14
2.1 Introduction.....	14
2.2 Factors Affecting Ship Speed	14
2.2.1 Significant Wave Height (Hs).....	14
2.2.2 Sea Current Speed	15
2.2.3 Wind Speed	16
2.2.4 Length-to-Beam Ratio (L/B).....	17
2.3 Prediction techniques.....	18
2.4 Limitations of Current Ship Speed Prediction Techniques	25
2.5 RESEARCH GAP	27
CHAPTER 3	30
RESEARCH METHODOLOGY	30
3.1 Introduction.....	30
3.2 RESEACRH APPROACH	32
3.3 System Architecture Overview	34
3.4 Data Acquisition	36

3.4.1 Measured Parameters	37
3.4.2 Sensor Assembly and Installation	37
3.4.3 Data Collection Process	40
3.4.4 IIUM Road Surface Testing	40
3.5 Data Preprocessing	41
3.6 Grey-Box Prediction Model	42
3.7 Real-Time Monitoring and Alert System	44
3.8 Experimental Setup.....	47
CHAPTER 4	50
RESULTS AND DISCUSSION	50
4.1 Introduction.....	50
4.2 Vessel Speed Analysis.....	50
4.3 Heatmap Correlation Analysis of Environmental Factors on Vessel Speed	53
4.4 Expected Grey-Box Prediction Results	55
4.5 Prototype Road Testing	56
4.6 Test 1: Maritime-Trained Model on Road Test (3 May 2026).....	57
4.6.1 Test Conditions and Hardware Performance	57
4.7 Test 2: Model Retrained on Test 1 Road Data (30 May 2026)	60
4.7.1 Retraining and Test Setup	60
4.7.3 Prediction Results After Retraining	61
4.8 Test 3: Combined Dataset — Maritime and Road Data (6 June 2026) ..	62
4.8.1 Training Strategy.....	62
4.8.3 Prediction Results	64
4.9 Performance Comparison Across All Tests.....	65
4.10 Real-Time Dashboard and Alert System Validation	66
4.11 Discussion of Key Findings.....	69
CHAPTER 5	72
CONCLUSION AND RECOMMENDATIONS	72
5.1 CONCLUSIONS.....	72
5.2 RECOMMENDATIONS	73
Reference	75
APPENDIX B	79
PROJECT CODE.....	79
APPENDIX A	98

DATASET	98
---------------	----

LIST OF TABLES

<i>Table 2.1: Summary of Ship Speed Prediction Models, Capabilities, and Limitations</i>	23
<i>Table 3.1: Dimensions of Main Hardware Components Used in the Prototype System</i>	38
<i>Table 3.2: Operational Logic of the Alert System</i>	45
<i>Table 4.1: Summary of Vessel Speed and High-Speed Events</i>	52
<i>Table 4.2: Expected Performance Metrics</i>	56
<i>Table 4.3: Grey-Box Model Performance Across All Evaluation Phases</i>	65
<i>Table 4.4: Alert State Distribution Across All Three Tests</i>	66

LIST OF FIGURES

<i>Figure 3.1 : General methodology flowchart</i>	31
<i>Figure 3.2 : Vessel operating route from Kuala Terengganu Jetty to the offshore oil rigs</i>	32
<i>Figure 3.3: Flowchart of the Overall Vessel Speed Monitoring System</i>	36
<i>Figure 3.4: Assembled vessel monitoring prototype and road-testing setup</i>	39
<i>Figure 3.5 :Prototype system architecture showing the hardware components, ESP32 IoT gateway with grey-box prediction model, and Blynk IoT dashboard</i>	39
<i>Figure 3.6: IIUM Campus Road Test Route used for prototype validation</i>	41
<i>Figure 3.7 : Flowchart of the Grey-Box Prediction Model for Vessel Speed</i>	44
<i>Figure 3.8: Interface of the Dashboard Used in the System</i>	46
<i>Figure 3.9: Wiring Diagram of the System Components</i>	49
<i>Figure 4.1: Vessel Speed Profile (April – July 2025)</i>	51
<i>Figure 4.2: Example of a speed snippet exceeding 20 knots</i>	53
<i>Figure 4.3: Correlation heatmap for April</i>	54
<i>Figure 4.4: Actual vs Predicted Vessel Speed</i>	56
<i>Figure 4.5: Test 1 — Measured SOG vs predicted speed (V_p).</i>	58
<i>Figure 4.6: Test 1 — Wind speed profile</i>	58
<i>Figure 4.7: Test 1 — Estimated significant wave height (s)</i>	58
<i>Figure 4.8: Test 2 — Measured SOG vs predicted speed (V_p)</i>	61
<i>Figure 4.9: Test 3 -Wind speed profile</i>	63
<i>Figure 4.10: Test 3 — Estimated significant wave height (H_s)</i>	63
<i>Figure 4.11: Test 3 — V_p vs SOG in knots</i>	64
<i>Figure 4.12: Test 3 — V_p vs SOG in km/h</i>	65

<i>Figure 4.13: GREEN Alert State ($V_p \geq 20$ knots)</i>	68
<i>Figure 4.14: YELLOW Alert State ($18 \text{ knots} \leq V_p < 20 \text{ knots}$)</i>	68
<i>Figure 4.15: RED Alert State ($V_p < 18$ knots)</i>	69

LIST OF ABBREVIATIONS

AIS	Automatic Identification System
ANN	Artificial Neural Network
DWT	Deadweight Tonnage
ESP32	Espressif 32-bit Microcontroller
ETA	Estimated Time of Arrival
GBM	Grey-Box Model
GPS	Global Positioning System
GPR	Gaussian Process Regression
Hs	Significant Wave Height
IQR	Interquartile Range
L/B	Length-to-Beam Ratio
LSTM	Long Short-Term Memory
ML	Machine Learning
PINN	Physics-Informed Neural Network
RF	Random Forest
RMSE	Root Mean Square Error
SOG	Speed Over Ground
STW	Speed Through Water
SVR	Support Vector Regression
XGBoost	Extreme Gradient Boostin

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

The shipping industry is a backbone of global trade, moving goods across oceans every day. One important factor in ship operations is speed, which affects fuel use, costs, and timely deliveries. However, ships are susceptible to environmental factors. Understanding how these environmental factors affect vessel speed can help operators plan better, save fuel, and improve safety.

With modern sensors, GPS, and Automatic Identification Systems (AIS), we can now collect a lot of data about ships and the seas they sail through. Machine learning makes it possible to take this data and find patterns that humans might miss. By using these tools, we can predict how fast a vessel will go under different conditions and even monitor it in real time, giving operators useful alerts if the ship is not performing as expected

1.2 PROBLEM STATEMENT

Accurate vessel speed prediction is essential for efficient and compliant maritime operations; however, many existing approaches insufficiently integrate environmental factors and vessel design characteristics, leading to models that do not fully reflect real operational conditions and therefore produce less reliable predictions

under varying environmental influences. In addition, current methods often rely either on physics-based models or purely data-driven techniques, with limited hybridization between the two: physics-based approaches offer interpretability but may lack accuracy in complex scenarios, while data-driven models provide flexibility but can yield physically inconsistent results, and this gap limits both predictive performance and physical realism. Furthermore, many studies are validated using restricted datasets or controlled environments, which reduces their applicability to real-world maritime operations, as the lack of extensive testing across diverse operational and environmental conditions limits model generalization and constrains their suitability for broader maritime deployment.

1.3 SCOPE OF PORJECT

This project focuses on understanding and predicting how environmental conditions affect vessel speed in real-world industrial operations. It looks specifically at wind speed, wind direction, and wave height as the main factors influencing ship performance.

The project will use machine learning techniques to analyze historical and real-time data, helping to build a model that can forecast vessel speed under different conditions. In addition, a real-time monitoring dashboard will be created to display vessel speed, provide predictions, and send alerts when the speed drops below a set threshold

Furthermore, the scope of this project includes prototype validation through road testing conducted at the International Islamic University Malaysia (IIUM) campus. In the absence of direct access to maritime field conditions at the prototype validation stage, road testing was selected as a practical and controlled alternative. The IIUM campus circuit, which spans approximately 2.7 kilometres per round and incorporates twelve road humps, provided a repeatable environment to evaluate sensor integration, real-time data acquisition, grey-box model performance, and dashboard alert functionality. Three iterative test sessions were conducted on 3 May 2026, 30 May 2026, and 6 June 2026, with progressive model retraining between sessions to adapt the system to the road environment.

1.4 OBJECTIVES

The objectives are:

1. To develop a machine learning model that can accurately predict vessel speed under different environmental conditions.
2. To implement a vessel speed compliant prototype with a real-time monitoring dashboard that displays vessel speed, forecasts, and threshold-based alerts.
3. To validate the functionality and performance of the machine learning and monitoring system.

CHAPTER 2

LITERATURE REVIEW

2.1 INTRODUCTION

Precise predicting of vessel speed is becoming more essential in modern maritime activities, as it directly impacts the estimated time of arrival (ETA), fuel usage, scheduling effectiveness, and overall safety in navigation. Before this, the historical data and simplified resistance theories were used to derive formulas and statistical analysis to estimate ship speed. Eventually, these traditional methods were developed into more advanced physics-driven models that were influenced by the ship-related and environmental factors. In today's world, with progress in digitalization, sensor technology, and comprehensive data collection systems like AIS and onboard monitoring tools, data-driven approaches and machine learning are increasingly used to enhance prediction accuracy in practical operating conditions.

2.2 FACTORS AFFECTING SHIP SPEED

2.2.1 Significant Wave Height (H_s)

Significantly, H_s is one of the environmental factors that affect ship speed and the overall performance of vessels. Waves add resistance and motions, such as heaving, pitching, and slamming, which require additional propulsion energy for the ship to

maintain a constant speed (Filtz et al., 2015). The magnitude of the wave resistance depends on wave height, wave period, and the orientation of the ship relative to the waves. For example, head seas generate more resistance and reduce speed more than following seas. Studies from observations and models report that moderate wave heights could reduce operational speed by 2-3 knots for cargo ships, changing the expected arrival times and increasing fuel consumption. The role of H_s increases under conditions of high deadweight tonnage or inappropriate hull design, becoming the most important environmental factor in speed forecasting and voyage scheduling J. Zhao et al. (2022). Thus, high accuracy in H_s prediction is critically needed for immediate speed optimization and energy-efficient route planning.

2.2.2 Sea Current Speed

Sea current speed has a strong influence on a vessel's SOG and voyage efficiency. It acts either in the direction of the ship's course, which is considered as following current, or acts against it, known as head current, thereby modifying the effective progress of the vessel without changing the engine output. Head currents increase the power required to maintain service speed, while favorable currents can reduce fuel consumption and the duration of the voyage. According to Babbedge, (1977), in practice, currents affect navigational decision-making because vessels are obliged to change their heading and rudder angle to compensate for cross-currents, increasing the hydrodynamic resistance and decreasing the effective speed of the vessel.

Moreover, the observations in the Pacific Ocean along the Japanese coast have shown that tidal and coastal currents cause measurable speed variations, especially in shallow waters and narrow fairways. As stated by the report *Evaluation of Ship Performance for Small-Sized Container Ship* (2022), sea currents are hence of great importance in operational ship performance and ETA predictions, especially in short-range navigation and coastal shipping operations, and need to be incorporated into both statistical and machine learning models for correct speed forecasting.

2.2.3 Wind Speed

The wind speed has a considerable effect on the speed of ships by creating aerodynamic resistance and influencing the formation of waves on the sea surface. Strong headwinds consequently increase drag on hull and superstructure, thus requiring higher engine power for a given speed, while crosswinds can induce a lateral drift that requires rudder corrections, with increased hydrodynamic resistance and energy consumption (Filtz et al., 2015). This factor will be even more pronounced for ships with large windage areas, like container ships or Ro-Ro ferries carrying cargo on deck. Empirical investigations have determined that speed losses due to wind may range between 0.5 and 2 knots, depending on vessel size, wind strength, and direction.

Moreover, it is stated that wind acting together with waves gives higher local wave height and choppy conditions, further enhancing the effects of resistance. Comparing it with other factors, wind is ranked as the second most important contributor to added resistance after wave height (J. Zhao et al., 2022)). Therefore, wind

is viewed as a highly significant factor for route optimization, voyage planning, and predictive speed modeling, especially concerning exposed coastal and open-sea conditions

2.2.4 Length-to-Beam Ratio (L/B)

The main design parameter that affects ship resistance, stability, and speed is the Length-to-Beam ratio (L/B). In general, a higher L/B means a slenderer hull, which reduces wave-making resistance and, therefore, offers the possibility of a more efficient propulsion at higher speeds. Moreover, a low L/B results in increased drag and, consequently, reducing speed performance Yuan et al. (2021). Optimal L/B ratios improve the flow patterns along the hull, reducing added resistance both in waves and during maneuvers in various hull form studies and bow entrance angle modifications of Ro-Ro ferries (Cahyadi Sugeng et al., 2022). But L/B interacts with other factors; for example, vessels of higher L/B are more sensitive to pitch and roll in high seas but enjoy the advantage of lower calm-water resistance. The formula for calculating L/B is:

$$\frac{L}{B} \quad (1)$$

Where L is the length of the ship in meters, and B is the beam or width in meters. Proper design of L/B is therefore highly significant for predicting vessel speed, fuel consumption, and overall performance under varying environmental conditions.

2.2.5 Deadweight Tonnage (DWT)

DWT is the total carrying capacity of a vessel, including cargo, fuel, crew, and provisions, which directly impacts the draught, displacement, and resistance of a ship. A heavier DWT increases the wetted surface area, creating a greater frictional resistance, and thus decreases achievable speed at the same engine output (Babbedge, 1977). This gets confirmed by full-scale voyage data showing that heavily loaded ships suffer more significant speed losses under similar sea conditions compared to lightly loaded vessels (Evaluation of Ship Performance for Small-Sized Container Ship, 2022). Moreover, DWT interacts with wave height, wind, and other environmental factors, amplifying the effect on speed and fuel consumption. The formula for DWT is given below:

$$DWT = Loaded Displacement - Lightweight Tonnage \quad (2)$$

Where displacement refers to the vessel's total weight in tones. Because DWT affects draft, resistance, and maneuverability, it is considered a highly significant structural and operational factor in predictive models for ship speed and voyage optimization.

2.3 PREDICTION TECHNIQUES

2.3.1 Physics-Informed Machine Learning (PIML)

Physics-informed machine learning integrates physical laws around ship motion, such as resistance theory and propulsion principles, into data-driven models to enhance prediction accuracy. Instead of pure reliance on datasets, PIML constrains the

model using established ship hydrodynamics and reduces the risk of unrealistic predictions. In the work by Lang, Wu, and Mao (2022), a physics-based speed–power relationship was first established by using calm-water resistance formulas, after which machine learning (e.g., XGBoost) was applied to compensate for environmental disturbances such as wave resistance and wind drag. This hybrid modelling approach enables more stable predictions, especially when operational data is limited or noisy.

A key advantage of PIML over other approaches is that it retains interpretability—it embeds physical insights like Froude number effects or added resistance in waves, as opposed to a neural network operating as a "black box." The effectiveness will depend on the quality of the physical assumptions. Inherently, any bias from wave, wind, or hull conditions not well captured by theoretical models will be passed on to the hybrid system. Despite this, PIML is finding increasing popularity because it offers a good compromise between accuracy and reliability. It is, therefore, well positioned to consider applications related to energy-efficient navigation and regulatory use.

2.3.2 Artificial Neural Networks (ANN)

ANN models are being widely applied in predicting ship speed as a function of a set of variables, usually containing engine power, draught, wave height, wind speed, and loading condition. For example, the studies by Bassam et al. (2023) and Tarelko & Rudzki (2020) showed the efficiency of feedforward ANN architectures in learning complex nonlinear relationships between ship speed and environmental/operational inputs without explicit usage of hydrodynamic equations. These models are particularly

useful for real-time onboard applications, such as speed recommendation, route planning, or fuel optimization.

However, ANNs require a vast amount of input data to perform well, and poor-quality datasets collected from few conditions may result in model overfitting and poor generalization. Such methods also suffer from the serious limitation of lacking transparency: ANN models hardly ever say why the ship would slow down under certain conditions, and for that reason, their use is doubted when safety-critical decisions are to be made. Despite these limitations, ANNs remain an important tool in maritime analytics because of their simplicity, robustness, and ability to work without prior knowledge of ship physics.

2.3.3 Long Short-Term Memory (LSTM) Networks

While traditional ANNs treat each input independently, LSTM networks capture temporal dependencies by learning from the patterns in sequence voyage data. Wang et al. (2022) used LSTM to predict vessel speed in inland waterways, feeding time-series data of previous speed, rudder angle, and current speed. Due to the fact that the motion of a ship is a time-dependent process and depends on the previous states (e.g., momentum, wave encounter cycles), LSTMs should be more suitable than static models for dynamic environments. They are particularly effective in predicting speed fluctuations during maneuvers, tidal changes, or varying sea states.

However, the weakness of LSTM models is that they need input data in large continuous datasets; usually, this information is obtained from sensors or through AIS logs, with problems concerning missing and noisy data. Also, LSTMs are computationally expensive and take more time for training than standard ANN models. Furthermore, similar to most deep learning models, they also lack physical interpretability. Because they handle the sequential behavior efficiently, they hold their importance in contemporary maritime intelligent systems.

2.3.4 Machine Learning Regression & Ensemble

These have been traditionally adopted in the prediction of ship speed because they can efficiently process multi-variable data. Examples include SVR, RF, XGBoost, and GPR, all of which have been compared to model the speed-power relationships by Bassam et al. (2022) using inputs such as wind, wave height, draught, and propeller RPM. The Random Forest and XGBoost ensemble models are particularly powerful since they combine several decision trees to reduce error and capture non-linear interactions among variables.

A key advantage of these methods is that they can give feature importance rankings to help identify which factors affect ship speed the most—for example, wave height or wind speed. They require less training time than deep learning models and, with smaller datasets, tend to perform better. However, they cannot model physical behavior outside the data range of training, and lack inherent understanding of physics, making them unreliable when the ships encounter extreme weather or other scenarios

they have not been trained on. Nevertheless, practicality remains with ensemble models due to their high accuracy and simplicity in implementation.

2.3.5 Grey-Box & Semi-Empirical Hybrid Models

Grey-box models fill the gap between purely theoretical hydrodynamic models and purely data-driven techniques. They normally use empirical resistance formulas for the prediction of calm-water speed, while applying either machine learning or statistical corrections to account for real-world perturbations such as waves or fouling. Moreira et al. (2021) developed a model that couples engine-propulsion equations with ANN-based residual correction in predicting fuel consumption under varying conditions. Similarly, Lang & Mao (2020) developed a semi-empirical formula for quantifying the speed loss in head seas using full-scale measurements.

These models have their advantages, as they are more interpretable compared to black-box neural networks, and operational data is needed much less. However, they strongly depend on how accurate the initial physics-based models are. If the resistance model ignores shallow waters, hull ageing, or the current's influence, then the residual learning part needs to compensate heavily, reducing effectiveness. Nevertheless, the use of grey-box models is quite common in naval architecture and energy efficiency studies because they are stable and adhere to the physical principles.

*Table 2.1: Summary of Ship Speed Prediction Models, Capabilities, and Limitations
(Adapted from Lang et al., 2022; Bassam et al., 2021; Abebe et al., 2022; Tarovik et al., 2023; Zhou et al., 2023; and others)*

Model type	Capabilities	Limitations	Citation
Grey-Box Model (GBM: PINN + XGBoost)	Integrates physics and data, robust with limited data, high accuracy	Needs physics-based model, more complex to implement	Lang et al., 2024
Generalized Regression Neural Network (GRNN)	Handles nonlinearities, robust to noise, best among tested ML models	Requires large, quality data; less interpretable	Yuan et al., 2021
Artificial Neural Network (ANN)	High accuracy, flexible, robust to missing data	Needs large datasets, risk of overfitting	-Bassam et al., n.d. -Tarelko & Rudzki, 2020 - Zhou et al., 2022
XGBoost (Extreme	Handles complex relationships,	Less interpretable, needs tuning	-Lang et al., 2022 - Lang & Mao, 2020

Gradient Boosting)	good with tabular data		
Random Forest Regression (RFR)	Handles nonlinearities, robust to outliers	Computationally intensive, less interpretable	-Abebe et al., 2020 - Zhou et al., 2022 - Lang & Mao, 2020
Gaussian Process Regression (GPR)	Provides uncertainty estimates, good with small data	Computationally expensive, less scalable	- Abebe et al., 2020) -Bassam et al., n.d.
Support Vector Regression (SVR)	Good with high-dimensional data, robust	Sensitive to parameter selection, less scalable	- Moreira et al., 2021 - Zhou et al., 2022 - Lang et al., 2022
Linear Regression	Simple, interpretable	Poor with nonlinearities, lower accuracy	-Bassam et al., 2022 -El Mekkaoui et al., 2023
LSTM (Long Short-Term Memory)	Captures temporal dependencies, good for time series	Needs large sequential data, complex tuning	- Wang et al., 2022 -El Mekkaoui et al., 2023

Semi-Empirical/Processes-Based	Physically interpretable, validated with experiments	Limited to specific conditions, less flexible	-Tarovik et al., 2024 - Lang & Mao, 2020
--------------------------------	--	---	---

2.4 LIMITATIONS OF CURRENT SHIP SPEED PREDICTION TECHNIQUES

Predicting ship speed under real sea conditions is a challenging task due to the complex interaction between environmental forces, vessel hydrodynamics, loading conditions, and operational decisions made by crew members. In contrast, while physics-based methods are interpretable, their primary simplifications regarding real-world conditions are in calm water, constant displacement, or linear wave resistance. Such simplifications are usually less reliable under very dynamic conditions relative to strong winds, irregular waves, shallow waters, or heavy marine fouling. Semi-empirical formulas for speed loss usually regard specific ships and thus cannot be generalized for different vessel types, hull forms, or ageing conditions; therefore, their use in state-of-the-art dynamic routing and energy efficiency systems is limited.

ANN, LSTM, Random Forest, and XGBoost are strong predictive models, but they have limitations in terms of interpretability and their dependency on data. These models represent "black boxes." Due to this, it becomes hard to trace the causality of a

prediction, which lowers the trust in safety-critical decision-making. Besides, they require large amounts of quality operational data, including speed logs, power measurements, wave height, wind speed, and current velocity. In fact, the outcome of ship sensors is noisy, missing, and biased data due to calibration errors or misalignment between onboard instruments and AIS data. Poor quality data during training may lead to overfitting or the generation of unreliable predictions on new voyages or on previously unknown routes.

Hybrid techniques, such as physics-informed machine learning and grey-box models, aim at incorporating physical laws in conjunction with data-driven learning but are still bounded by the accuracy of the embedded physical assumptions. If the imbedded physics does not account for nonlinear effects in oblique waves, shallow water effects, or propeller ventilation, then this has to be compensated by the machine learning part, often introducing approximation errors. These hybrid systems also require expert knowledge in both naval architecture and data science. Implementation is more complex and hence not as accessible for small shipping companies. Such models are also usually vessel-specific and need recalibration when ships with different hull shapes, loading conditions, or different operational profiles are modeled.

Another important limitation of most approaches is that there does not exist a standardized dataset for model validation. Most studies rely on data from a single vessel or limited sea area, making it hard to compare model performance under different sea conditions or ship types. Real-time deployment introduces several computational challenges; some methods using deep learning may need computational power and

might not be suitable for limited onboard hardware. In addition, environmental uncertainties-such as rapid changes in sea state, tropical cyclones, or even inaccuracies in weather forecasts-can substantially reduce the accuracy, particularly for long-distance ETA prediction or fuel planning. In conclusion, no individual technique works universally across all possible maritime conditions despite the progress achieved so far. Physics-based models are interpretable but cannot capture real-time variability; data-driven models are adaptable but need to be more transparent; and hybrid methods improve accuracy but generally demand high expertise and quality datasets. Limitations with regard to robustness, generalization ability, and explain ability of a wider range of environmental uncertainty, vessel-specific behavior, and real-time optimization point to the need for better prediction systems.

2.5 RESEARCH GAP

Despite numerous studies conducted on ship speed prediction and performance modeling, significant gaps are evident regarding the complex interaction between environmental and design parameters under real-world operating conditions. These early empirical analyses, such as those by Babbedge (1977) and Robinson (1975), provided initial and valuable insights into understanding ship voyage data and speed measurement. However, they had limitations in the generalization within various sea states and vessel types. In a similar way, research efforts like Filtz et al. (2015) and X. Zhao et al. (2025) have indicated how environmental factors-especially wind speed, sea current, and wave height-affect ship resistance and speed. Yet, much of their

frameworks were limited to controlled datasets and simplified hydrodynamic assumptions.

However, the emergence of machine learning and data-driven modeling has opened new horizons for ship performance prediction. Several models, such as artificial neural networks (Bassam et al. (2023); Tarelko & Rudzki (2020)), LSTM networks (Wang et al. (2022)), and hybrid or physics-informed approaches (Lang et al. (2022); Schirrmann et al. (2022)), have been proposed to predict ship speed or propulsion power with good accuracy. However, these studies often put much more emphasis on algorithmic optimization than on physical interpretability. The outcome of such an approach might be models that statistically perform well, though they may also fail to reflect the hydrodynamic reality of dynamic ocean conditions. Similarly, although some research addressed environmental variability (Abebe et al. (2020); Zhou et al. (2022)), there is a clear deficiency of research at the moment with respect to integrating multiple environmental variables, including significant wave height, wind speed, and sea current, into a unified modeling framework.

Moreover, the literature on speed optimization for ships has focused on energy efficiency and environmental sustainability, such as the works by Wei & Deng. and Tarovik et al., but often lacks comprehensive validation with diverse datasets from real sea areas. The papers often refer to small or region-specific samples, for example, Japanese coastal or Arctic datasets, which will make generalization difficult to realize for global shipping routes with more heterogeneous environmental conditions. This lack

of generalized datasets restricts the scalability and reliability of predictive models in practical maritime operations.

Overall, the literature reveals three major gaps:

1. Incomplete integration of environmental and design factors in predictive models.
2. The lack of sufficient hybridization between physics-based and data-driven methods to ensure both accuracy and physical interpretability.
3. Limited validation across different real-world conditions restricts generalization for various global maritime applications.

From this perspective, addressing these gaps would contribute to developing a more robust, interpretable, and energy-efficient ship speed prediction model that could support ETA estimation and operational decision-making for modern digital and green shipping.

CHAPTER 3

RESEARCH METHODOLOGY

3.1 INTRODUCTION

This chapter outlines the methodology for developing the vessel speed monitoring and prediction system, taking into account environmental effects. The overall workflow is summarized in the general methodology flowchart (Figure 3.1), which shows the main stages from confirming objectives and literature review to design, fabrication, integration, testing, and validation. The project is currently focused on the design of the prototype and the development of the machine learning model, which form the core of the system.

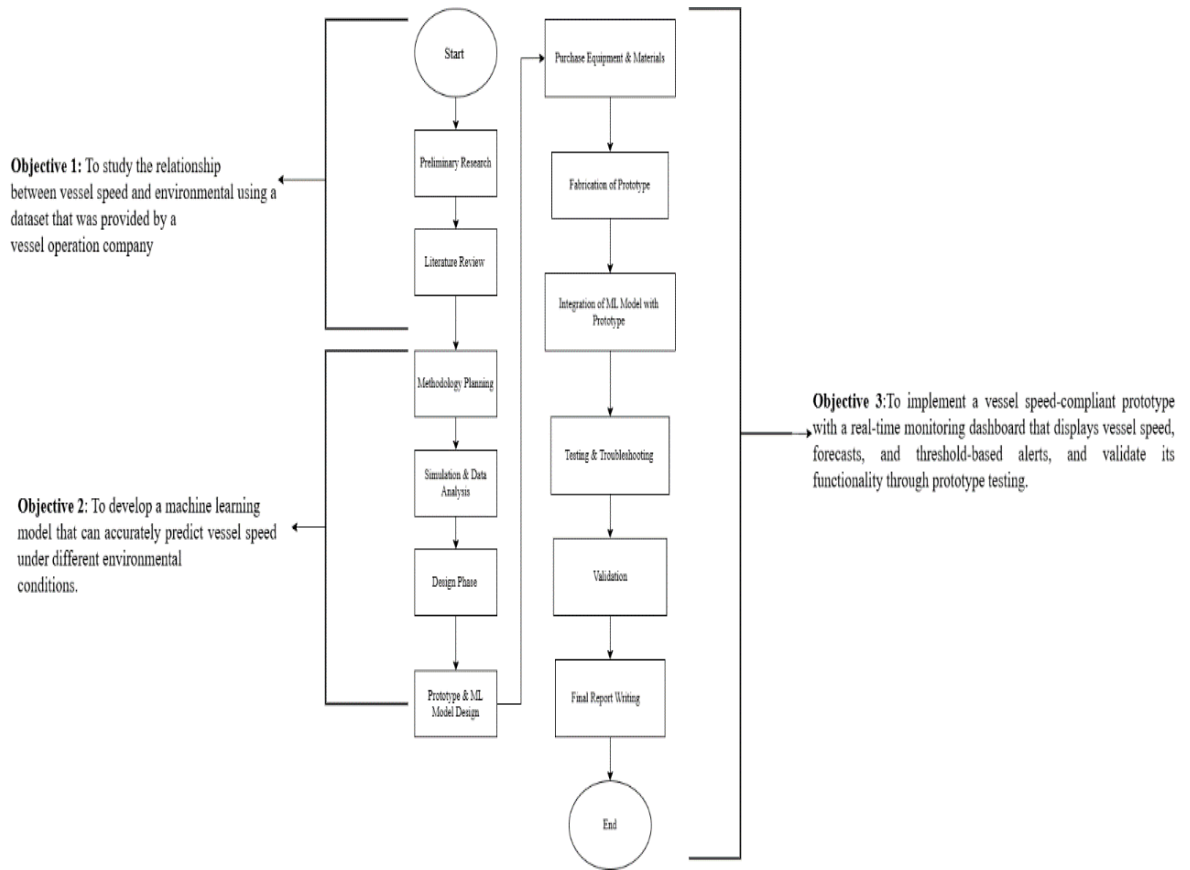


Figure 3.1 : General methodology flowchart

The proposed system focuses on real-time monitoring and prediction of vessel speed using key environmental factors, namely wind speed and wave height. A grey-box machine learning approach, combining Physics-Informed Neural Networks (PINN) and XGBoost, is implemented to improve prediction accuracy while maintaining physical interpretability. The flowchart highlights all research and development steps, including prototype and ML model design, equipment procurement, fabrication, integration, testing, troubleshooting, and final validation.

3.2 RESEACRH APPROACH

The datasets used in this research were collected from the vessel Jati 6 by a vessel operating company during operations along the route from KT Jetty to the offshore oil rig, with an approximate one-way travel time of six hours. Data were recorded over four months, from April to July, covering multiple voyages under varying operating and environmental conditions. All data were logged in the UTC+8:00 time zone. The operational route of the vessel is shown in Figure 3.2, which illustrates the trajectory from the jetty to the rig with approximate route of 181 km.

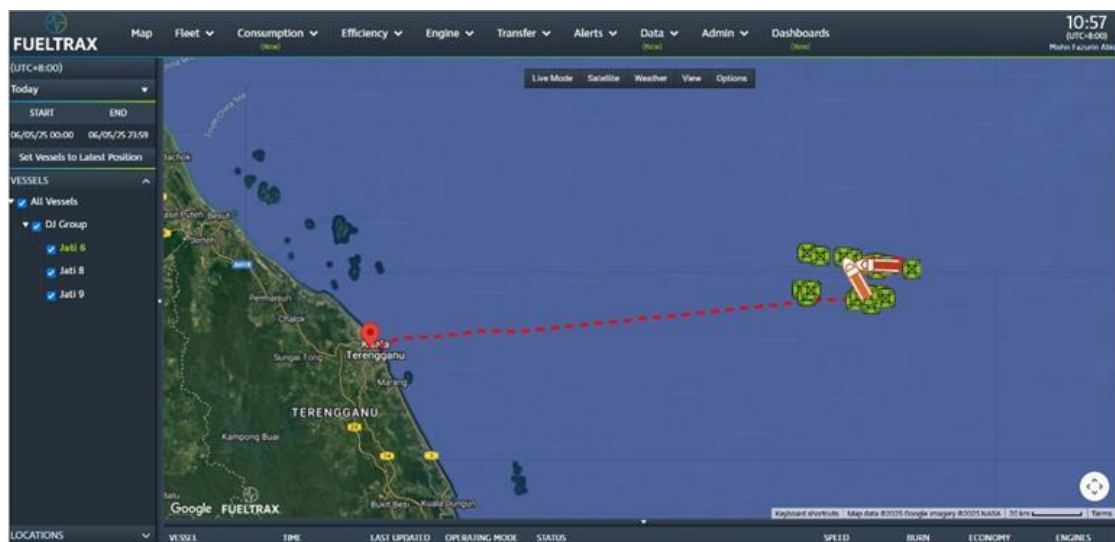


Figure 3.2 : Vessel operating route from Kuala Terengganu Jetty to the offshore oil rigs

The dataset includes navigational data (latitude, longitude, vessel speed, and heading), operational data (operating mode, economy mode, and fuel consumption), and environmental parameters (significant wave height, wind speed and direction, and

temperature). These data support vessel speed monitoring, threshold-based alerting, and grey-box machine learning analysis. The snippet dataset is provided in Appendix B.

Historical operational datasets from a vessel operating company were analyzed to examine the relationship between vessel speed and environmental factors. Data were cleaned and filtered to remove non-transit periods and erroneous readings, and statistical correlation analysis was performed to assess the influence of environmental conditions on vessel performance.

A machine learning prediction model was then developed using the processed data. Key environmental and vessel parameters were used as input features, normalized for consistency, and modeled with XGBoost regression to capture nonlinear relationships. Model performance was evaluated using MAE, RMSE, and R^2 metrics to ensure accurate vessel speed predictions under varying operational conditions.

The research will adopt a system-based, experimental approach to analyze the relationship between vessel speed and environmental factors through measurements and machine learning models. This approach will combine real-time measurement, threshold monitoring, and grey-box modeling to ensure reliable operation while enabling meaningful performance assessment. Overall, the research process will aim to mirror an industrial monitoring system.

Firstly, environmental and vessel operating data will be obtained in real time through sensors mounted on the vessel. This may include wind speed and wave height. Vessel speed will then be continuously monitored by a microcontroller and compared

to predefined operating thresholds. As a result, the warning system will rely solely on physical measurements collected from the sensors.

Next, the collected data will be transmitted to a processing unit for further analysis. A grey-box machine learning algorithm will be used to predict vessel speed under various environmental conditions. These predictions will help verify vessel speed data, study the effects of different conditions on vessel operation, and evaluate the accuracy of the vessel monitoring system.

Finally, an analytical dashboard will be implemented for real-time visualization of vessel speed, environmental parameters, and alert information. This research method will enable both the implementation and assessment of the proposed system in accordance with the objectives of vessel speed monitoring, prediction, and verification under varying environmental conditions.

3.3 SYSTEM ARCHITECTURE OVERVIEW

The overall system architecture of the proposed vessel speed prediction and monitoring system is illustrated in Figure 3.3. The system is designed using a layered approach to ensure modularity, real-time performance, and reliability. It integrates environmental sensing, embedded processing, machine learning prediction, and an alert mechanism.

At the Sensing Layer, various sensors will acquire measurements of vessel and environmental parameters, including wind speed and wave height. The sensor data will

be collected continuously in real time, reflecting the operational conditions of the vessel.

The Processing Layer will be implemented using a microcontroller responsible for data acquisition, time synchronization, pre-processing, and execution of monitoring logic. Raw sensor data will be filtered and validated to ensure consistency before further analysis.

The Prediction Layer will use a grey-box model, combining a Physics-Informed Neural Network with an XGBoost machine learning model. It will process environmental parameters and derived features to estimate vessel speed, which will be used for performance evaluation and model validation against measured data.

Finally, the Monitoring and Alert Layer will continuously compare predicted vessel speed against predefined operational thresholds. If abnormal conditions are detected, this layer will trigger visual and audible alerts for the operator. Sensor readings, environmental conditions, and system status will be displayed on a dashboard interface to support real-time situational awareness.

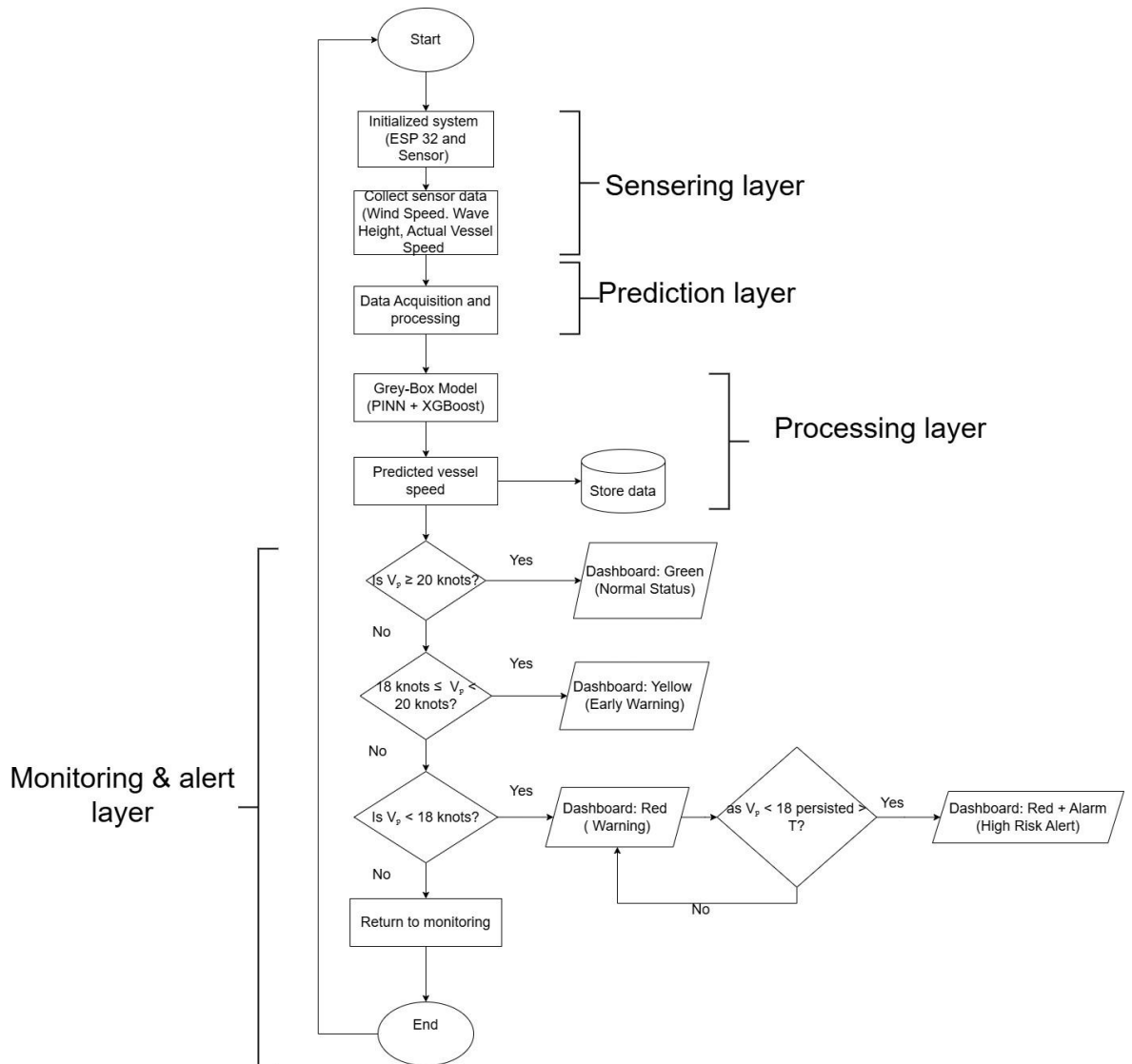


Figure 3.3: Flowchart of the Overall Vessel Speed Monitoring System

3.4 DATA ACQUISITION

The data acquisition process was critical for obtaining reliable measurements of vessel performance under environmental influences. This study collected real-time data from multiple sensors installed on the vessel, providing continuous monitoring of both environmental conditions and vessel speed.

3.4.1 Measured Parameters

The following parameters were recorded during vessel operation:

- Wind Speed – measured by the Airmar 200WX Multisensor Station to account for aerodynamic resistance.
- Wave Height – measured by the BNO085 9-DOF IMU sensor to capture hydrodynamic disturbances.

These parameters were selected because they directly affected vessel speed and propulsion performance and were essential for both prediction and validation purposes. Based on contractual obligations, the vessel was required to maintain a minimum transit speed of 20 knots between the jetty and the oil rig. Operating below this threshold could result in a Notice of Default (NOD) and potential penalties from the client. This requirement was reflected in the collected datasets and served as the basis for the speed threshold used in the real-time alert system.

3.4.2 Sensor Assembly and Installation

The vessel monitoring prototype consisted of an ESP32 microcontroller, BNO085 IMU sensor, Airmar 200WX Weather Station, and supporting communication and power modules. The ESP32 and IMU sensor were installed inside the enclosure housing, while the Airmar 200WX sensor was mounted externally to ensure accurate environmental measurements.

The enclosure was designed to provide protection for the internal electronic components while maintaining a compact and portable configuration suitable for both laboratory and field testing. The external mounting arrangement of the Airmar sensor allowed unobstructed wind measurements during data collection.

Table 3.1: Dimensions of Main Hardware Components Used in the Prototype System

Component	Dimensions
Housing Unit	203.2 mm × 152.4 mm × 76.2 mm
BNO085 IMU Sensor	25.4 mm × 17.8 mm × 2.5 mm
Airmar 200WX Weather Station	Approximately 702 mm × 85 mm × 72 mm
Power Supply Unit (PSU)	199 mm × 98 mm × 38 mm

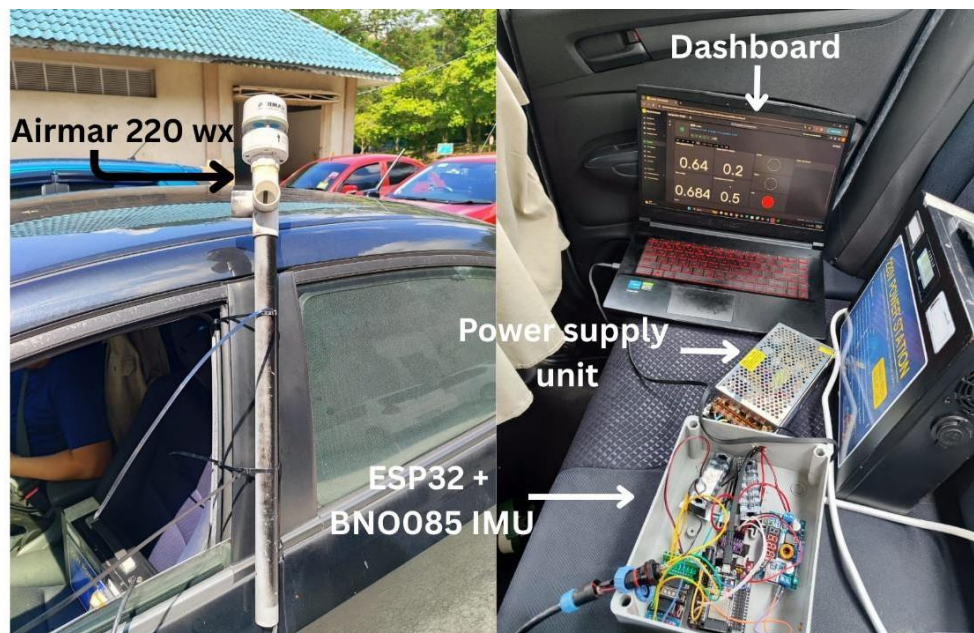


Figure 3.4: Assembled vessel monitoring prototype and road-testing setup

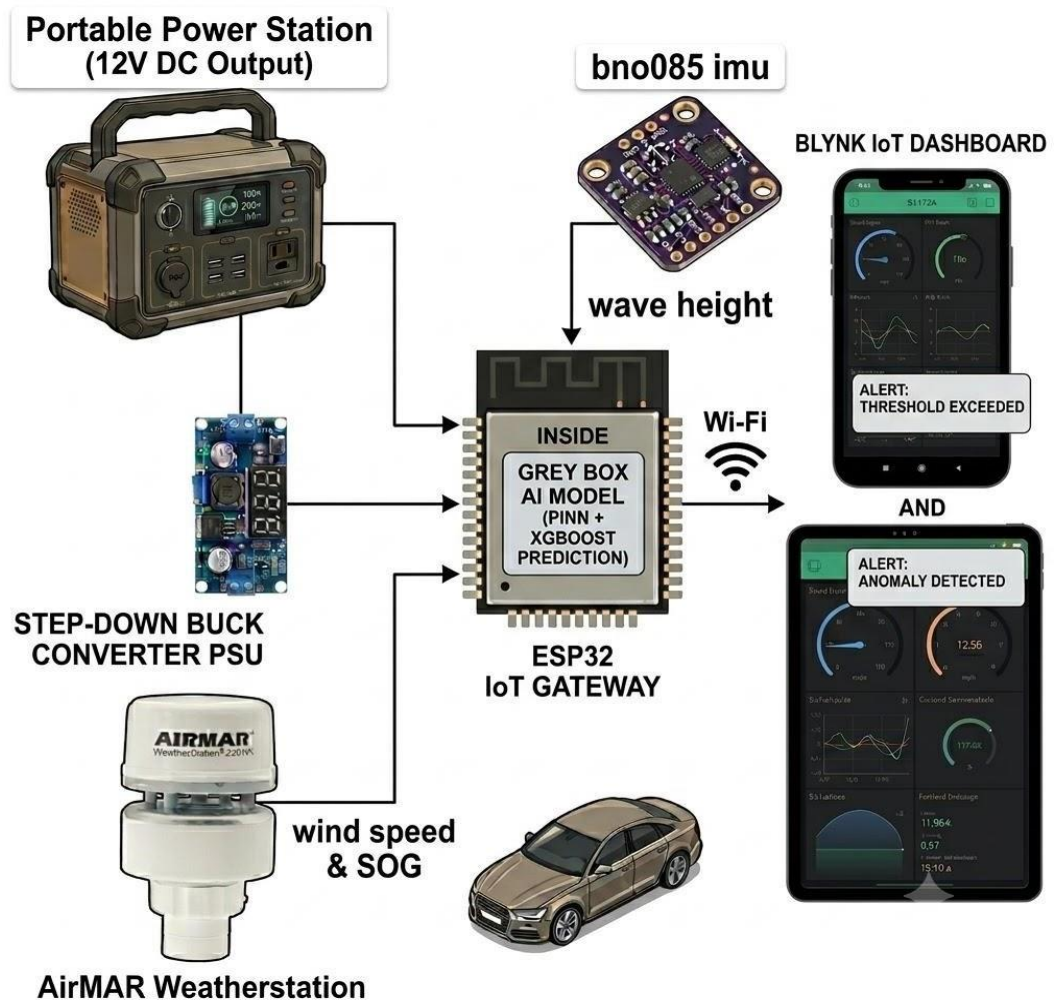


Figure 3.5 :Prototype system architecture showing the hardware components, ESP32 IoT gateway with grey-box prediction model, and Blynk IoT dashboard

3.4.3 Data Collection Process

The data collection process was conducted in two stages to evaluate the functionality and reliability of the proposed vessel monitoring system under different operating conditions.

3.4.4 IIUM Road Surface Testing

Initial testing was conducted within the IIUM area using a vehicle-based road surface experiment to validate sensor communication, data synchronization, and real-time data acquisition performance. During the experiment, the Airmar 200WX sensor was mounted externally at the side of the vehicle door and secured using cable ties to ensure stable environmental measurements during movement. The enclosure containing the ESP32 and IMU sensor was placed inside the back seat of the vehicle.

A total of four testing rounds were conducted around the IIUM route, where each round covered approximately 2.7 km. The selected route contained approximately 12 road humps, which were used to simulate vessel motion and wave disturbances for evaluating the IMU response and wave estimation capability.

The road surface testing focused on:

- Validation of system functionality
- Troubleshooting hardware and software integration

- Verification of sensor communication and synchronization
- Real-time dashboard monitoring performance
- Evaluation of IMU response to motion disturbances
- Stability of real-time data acquisition and transmission

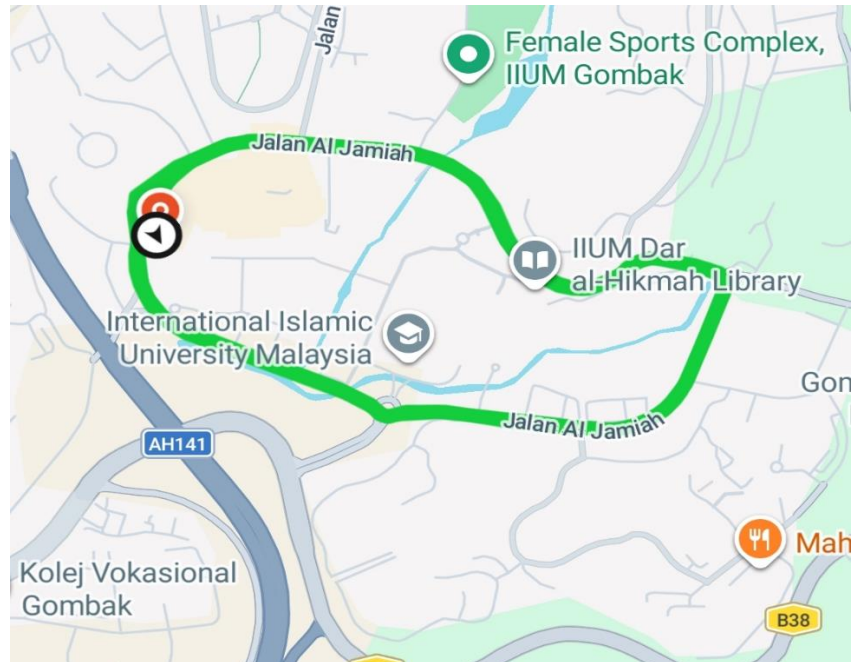


Figure 3.6: IIUM Campus Road Test Route used for prototype validation

3.5 DATA PREPROCESSING

Before the sensor data were used for prediction and monitoring, the raw measurements underwent a preprocessing phase to improve accuracy, reliability, and completeness. Data cleaning involved the removal of irrelevant or misleading data, including GPS errors, negative or extreme speed readings, and repeated measurements caused by sensor failures. Outliers in vessel speed, wind speed, and wave height were

filtered using statistical methods such as the Interquartile Range (IQR) filter and threshold-based filtering techniques, while minor variations in the data were smoothed to reduce noise.

Environmental sensor data were timestamped by the ESP32 to ensure synchronization with vessel speed measurements. Continuous variables, including wind speed and wave height, were normalized to allow efficient use by the machine learning model regardless of differences in units or value ranges. This preprocessing pipeline ensured that all sensor readings were cleaned, synchronized, and normalized, thereby providing meaningful input for the grey-box model and enabling the system to focus on actual conditions during vessel transit.

3.6 GREY-BOX PREDICTION MODEL

Grey-box modeling was used to balance interpretability and prediction accuracy by combining physics-based reasoning with data-driven learning for vessel speed prediction.

The physics-informed component captured the general relationship between environmental conditions and vessel speed, where higher wind speeds and wave heights resulted in increased aerodynamic and hydrodynamic resistance, leading to lower vessel speeds. Instead of implementing complex hydrodynamic equations, simplified physical relationships were embedded into the model to guide learning and constrain predictions within physically realistic bounds.

For the machine learning component, XGBoost regression was applied to efficiently model nonlinear relationships in structured sensor data. Inputs to the model included wind speed, wave height, and Speed Over Ground (SOG). The output of the model was the predicted vessel speed, which was compared against measured values for validation purposes.

This grey-box model leveraged both domain knowledge and data-driven optimization by combining physics-informed relationships with XGBoost regression. The physics component ensured physically plausible predictions, while XGBoost captured subtle nonlinear interactions between environmental factors and vessel performance. The dataset was divided into training and testing sets, and model performance was evaluated using Root Mean Square Error (RMSE) and the coefficient of determination (R^2) metrics to ensure reliability and accuracy.

The integrated PINN + XGBoost grey-box framework provided a robust predictive tool for real-time monitoring, supporting the alert system and enabling enhanced analysis of environmental influences on vessel speed.

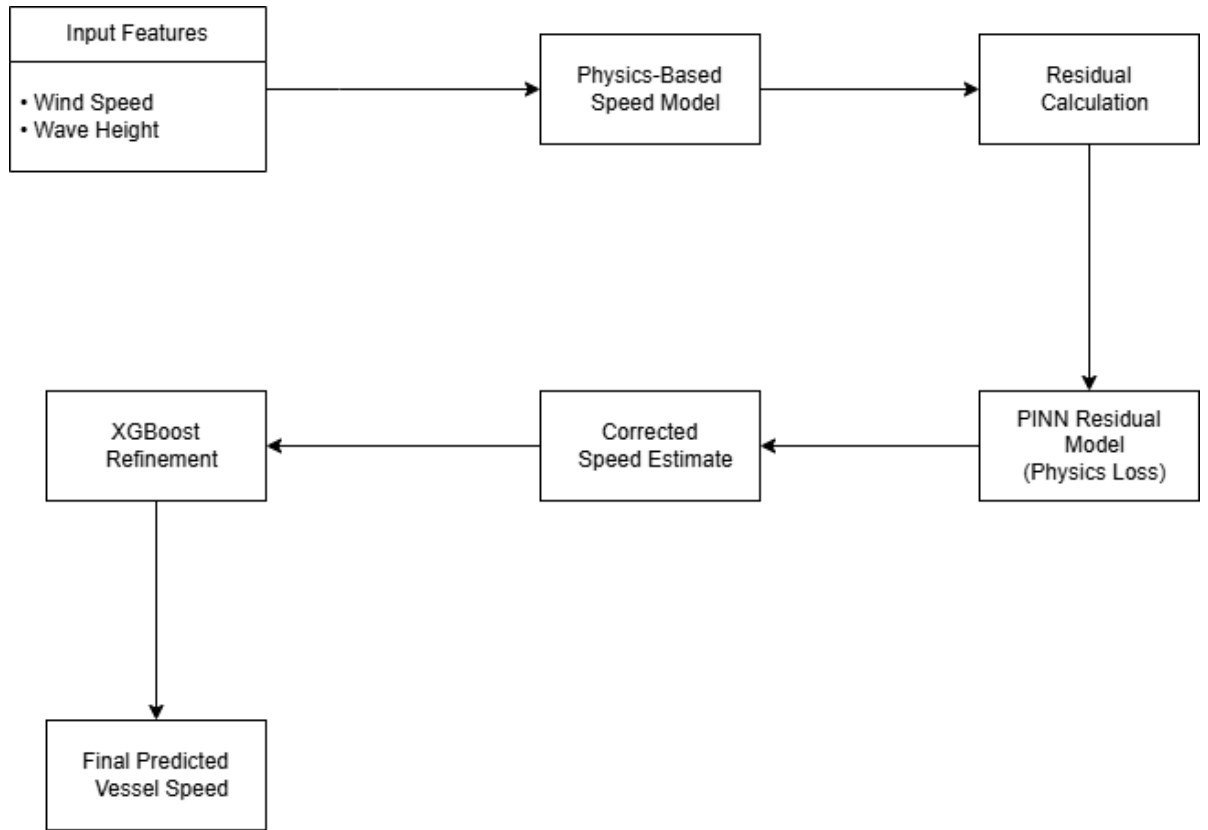


Figure 3.7 : Flowchart of the Grey-Box Prediction Model for Vessel Speed

3.7 REAL-TIME MONITORING AND ALERT SYSTEM

The real-time monitoring and alert system continuously monitored vessel speed and environmental conditions to maintain operational safety and efficiency. Sensor data, including Speed Over Ground (SOG), wind speed, and wave height, were collected and processed in real time using the ESP32 microcontroller. The processed data were subsequently used for both monitoring and prediction purposes.

Visual and auditory alerts were implemented through a real-time dashboard interface. Real-time vessel speed measurements, environmental parameters, and alert information were displayed on the dashboard to provide continuous operational awareness. The alert logic is summarized in Table 3.1, which shows the relationship between vessel speed conditions and system responses.

Table 3.2: Operational Logic of the Alert System

Predicted Vessel Speed	Condition	Alert Action
$V \geq 20$ knots	Vessel speed is within normal operating range	Green indicator
V close to 20 knots	Vessel speed approaching the minimum threshold	Yellow indicator
$V < 20$ knots	Vessel speed below the minimum threshold	Red indicator
$V < 20$ knots for $>T$ limit	Prolonged operation below threshold	Red indicator and audible buzzer

The alert thresholds were based on the contractual requirement that the vessel maintain a minimum transit speed of 20 knots between the jetty and the oil rig. Speeds approaching the threshold provided early warning indications, while prolonged operation below 20 knots could result in a Notice of Default (NOD) and potential penalties. Under these conditions, both visual and audible alerts were activated to notify operators.

The overall system process involved several stages, including sensor data acquisition, data preprocessing, grey-box prediction, threshold comparison, alert activation, dashboard updating, and data storage. A flowchart (Figure 3.1) illustrates these processes to provide a clearer understanding of the data flow during real-time system operation.



Figure 3.8: Interface of the Dashboard Used in the System

Additionally, a dashboard interface (Figure 3.6) was used to display vessel speed, environmental values, and warning messages in real time. This interface supported improved monitoring, understanding, and management of system operations.

By combining real-time measurements with predictive models, the system provided instant operational warnings and supported data-driven analysis for performance optimization while maintaining the physical interpretability of the predictions.

The experimental design involved testing the proposed vessel speed prediction and alert system through implementation on a vessel operating along a fixed route between the pier and the oil rig.

3.8 EXPERIMENTAL SETUP

The hardware architecture was centered around the ESP32, which served as the high-speed central processing and data aggregation unit. The ESP32 facilitated real-time data acquisition from multiple marine sensors and managed the operational logic of the local alert system.

Environmental and navigational data were measured using the high-precision Airmar 200WX Weather Station. This multisensor system provided measurements of wind speed and Speed Over Ground (SOG). Data communication between the Airmar 200WX and the ESP32 microcontroller was achieved through a wired NMEA 0183 interface with RS-232 signal conversion. Hydrodynamic events and wave motion were measured using the Adafruit BNO085 9-DOF IMU sensor.

Power management was handled using an industrial AC/DC power supply that converted 220V AC into a stabilized DC rail (12V/24V). This stabilized DC supply powered the industrial tower light and the Airmar 200WX sensor. An adjustable DC-DC buck converter further reduced the voltage to 5V to power the ESP32 and supporting low-voltage components.

The ESP32 executed a preprocessing pipeline that included noise filtering, GPS-based time synchronization, and geospatial filtering. To focus specifically on vessel performance during transit, the system isolated data within a predefined “Pier-to-Rig” corridor. Using latitude and longitude coordinates, only data collected during vessel transit were retained, while measurements recorded during docking, idling, or harbor maneuvers were excluded.

After local processing, the data were transmitted from the ESP32 to a laptop computer via USB communication for further analysis. The laptop platform executed a grey-box prediction model combining theoretical ship resistance physics with an XGBoost residual-learning approach to estimate vessel speed.

The alert mechanism operated based on predefined operational logic conditions. Real-time sensor values, predicted vessel speed, and alert statuses were displayed on a central monitoring dashboard to provide continuous operational awareness.

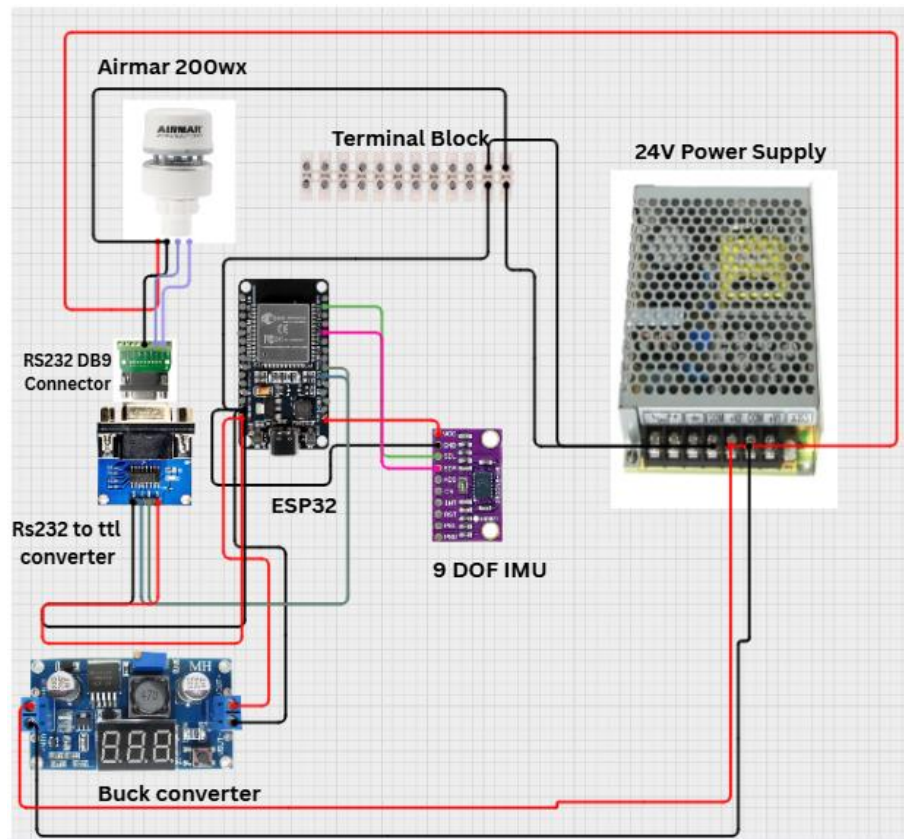


Figure 3.9: Wiring Diagram of the System Components

CHAPTER 4

RESULTS AND DISCUSSION

4.1 INTRODUCTION

This chapter presents the results of the vessel speed analysis, correlation assessment, grey-box prediction model evaluation, and prototype system validation. The first part of this chapter (Sections 4.2–4.4) uses operational data provided by a vessel operating company, covering four months of actual vessel transits from April to July. The purpose is to examine vessel speed behaviour, understand the influence of environmental conditions such as wind speed and wave height, and evaluate the accuracy of the physics-guided grey-box prediction model.

The second part of this chapter (Sections 4.5–4.9) presents the results from the physical prototype developed and validated through road testing. This includes hardware and sensor validation, real-time model performance on prototype-collected data, and verification of the dashboard alert system. A unified discussion in Section 4.11 consolidates findings from both phases and contextualises them against the three project objectives.

4.2 VESSEL SPEED ANALYSIS

The vessel is required to maintain a minimum operational speed of 20 knots to meet transit schedules, ensure operational efficiency, and comply with mission or contractual requirements. Continuous monitoring of vessel speed is essential to verify compliance and to allow timely corrective actions in case of deviations.

From the speed profiles (Figures 4.1) indicates that, for most of the reporting period, the vessel operated below the 20-knot target. Despite this, there were brief periods when the vessel reached or exceeded 20 knots, demonstrating that the vessel is capable of achieving operational targets under favorable conditions. A notable example occurred in May, when the vessel-maintained speeds above 20 knots for a duration of 12 minutes, reaching An average speed of 23.03 knots (Figure 4.5). This instance highlights that target speeds can be attained, although sustained operation above 20 knots was limited.

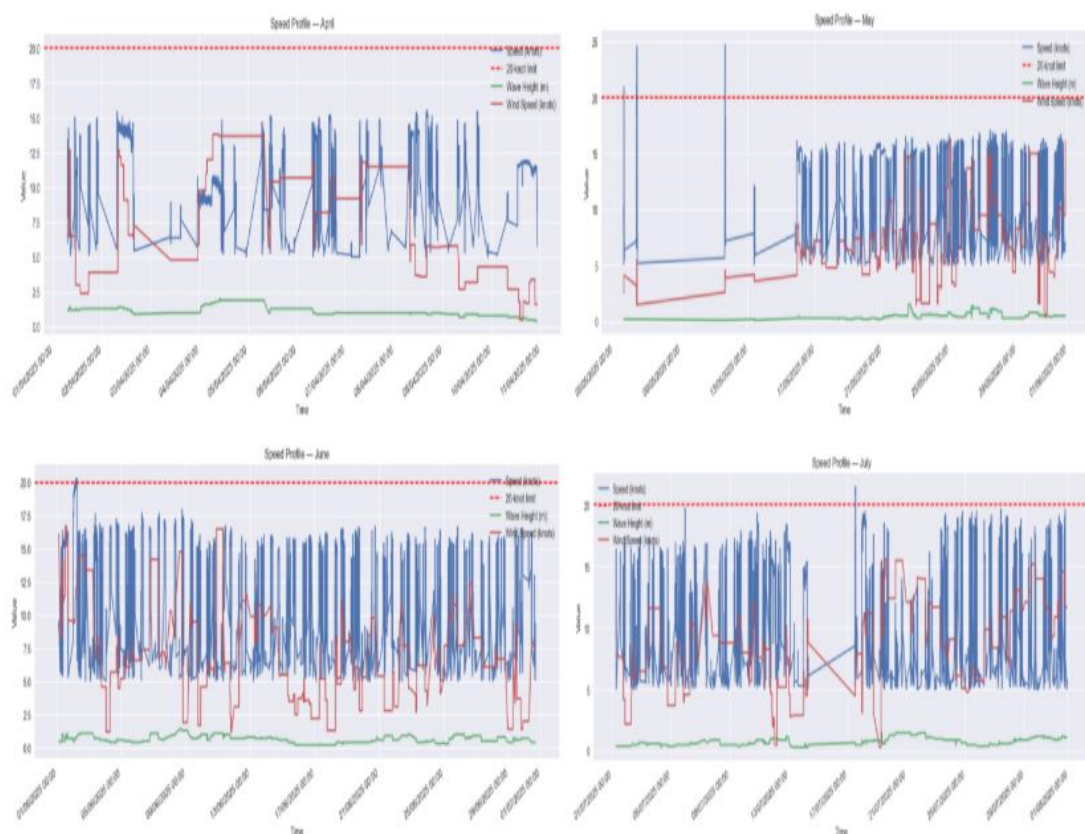


Figure 4.1: Vessel Speed Profile (April – July 2025)

The summary of vessel speed and events exceeding the 20-knot target is presented in Table 4.1:

Table 4.1: Summary of Vessel Speed and High-Speed Events

Month	Average Speed (knots)	Max Speed (knots)	No. of >20 kt Events	Total Duration >20 kt (min)
April	11.64	15.54	0	0.0
May	13.53	24.71	8	48.0
June	13.23	20.34	31	13.0
July	13.22	21.52	1	7.0

From the table, it is evident that:

- April: The vessel did not reach the 20-knot target.
- May: Achieved the highest peak speed and multiple brief periods above 20 knots.
- June: Recorded the most frequent exceedances, though most were brief.
- July: Only one short period above 20 knots was observed.

Overall, the vessel operated below the required 20-knot speed for the majority of the period. While occasional brief exceedances occurred, consistent compliance with the operational target was limited. These findings highlight the need for operational adjustments and continuous speed monitoring to ensure that the vessel meets the required transit performance and efficiency.

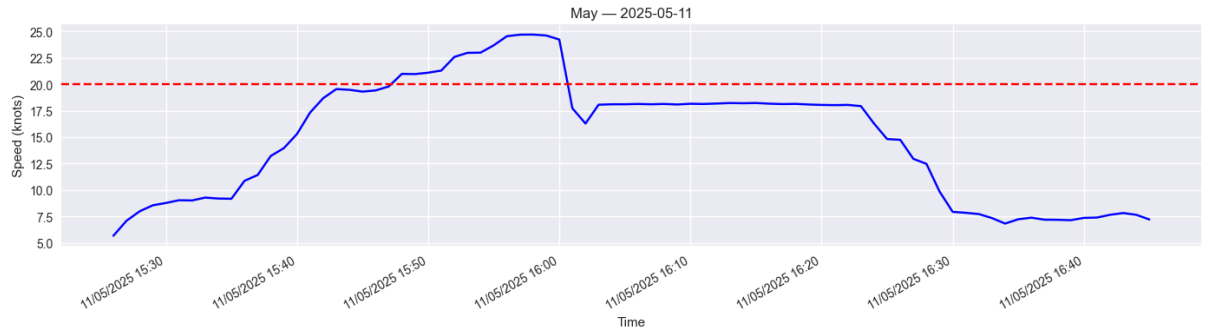


Figure 4.2: Example of a speed snippet exceeding 20 knots

4.3 HEATMAP CORRELATION ANALYSIS OF ENVIRONMENTAL FACTORS ON VESSEL SPEED

The objective of this analysis is to investigate how environmental factors influence vessel speed. Understanding these effects helps optimize vessel performance and predict operational behavior under varying sea conditions.

Correlation analysis was performed between vessel speed (VESSEL_KNOTS) and environmental parameters. The Pearson correlation coefficients are visualized using a heatmap for the month of April, the period showing the most notable environmental impact, with values expressed to two decimal places.

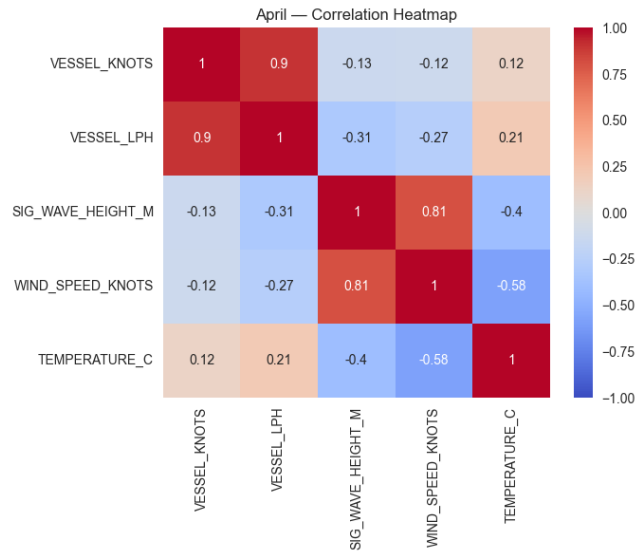


Figure 4.3: Correlation heatmap for April

In April, vessel speed exhibited weak negative correlations with wave height (-0.13) and wind speed (-0.12), indicating that higher waves and headwinds slightly reduced vessel speed. Temperature showed a weak positive correlation (0.12), suggesting a negligible influence on speed. Overall, environmental factors had only a minor impact, with vessel speed being primarily governed by operational control.

The analysis indicates that wave height and wind can slightly reduce vessel speed, while temperature has negligible effect. Selecting the most representative month for visualization simplifies interpretation while highlighting the environmental influence on vessel performance.

4.4 EXPECTED GREY-BOX PREDICTION RESULTS

A hybrid grey-box prediction model combining a Physics-Informed Neural Network (PINN) with XGBoost was implemented to predict vessel speed using the four-month historical operational dataset. By combining these methods, the model benefits from both physical interpretability and machine learning flexibility, enabling accurate prediction under varying environmental and operational conditions.

The predicted vessel speed closely follows the measured values, effectively capturing both steady-state behaviour and sudden speed fluctuations, including periods when the vessel temporarily exceeds or drops below the 20-knot target.

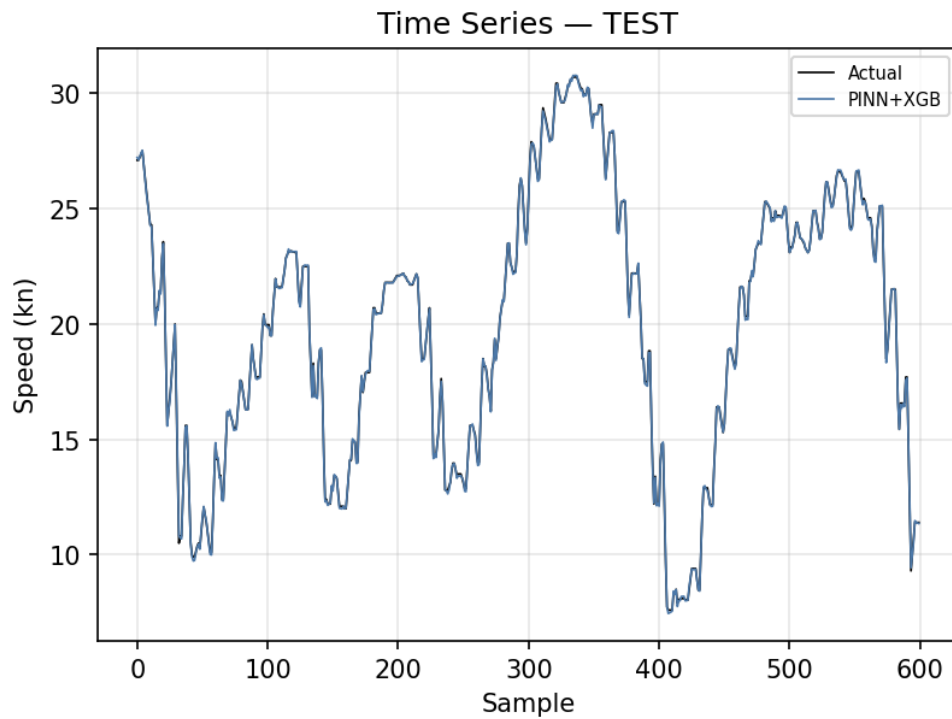


Figure 4.4: Actual vs Predicted Vessel Speed

Table 4.2: Expected Performance Metrics

Split	RMSE (kn)	MAE (kn)	R ²	±1 kn	±2 kn
TRAIN	0.0941	0.0568	0.9998	100.0%	100.0%
VAL	0.0743	0.0545	0.9999	100.0%	100.0%
TEST	0.2462	0.1057	0.9991	97.9%	100.0%

4.5 PROTOTYPE ROAD TESTING

The predictions were streamed in real time to a Blynk IoT dashboard with a three-state alert system (GREEN / YELLOW / RED). Each test used a different training dataset for the grey-box model:

- Test 1: The model was trained exclusively on the historical Jati 6 maritime vessel dataset. The purpose was to verify the system functionality and observe maritime-model behavior in a road environment.
- Test 2: Model retrained on road data collected from Test 1. Purpose was to determine whether domain-matched resolves the maritime–road mismatch that was obtained in Test 1.
- Test 3 Model retrained on a combination of the Test 1, and Test 2 road data. Purpose was to assess whether broader training coverage maintains accuracy while improving robustness across varying conditions.

4.6 TEST 1: MARITIME-TRAINED MODEL ON ROAD TEST (3 MAY 2026)

4.6.1 Test Conditions and Hardware Performance

The first prototype road test was conducted on 3 May 2026 on the IIUM campus circuit, running for 22.6 minutes and collecting 12,683 samples at a high sample rate. The vehicle reached a maximum speed of 35.3 knots (65.4 km/h). The route included 12 speed humps to simulate vessel motion disturbances. The model deployed was the grey-box PINN + XGBoost trained exclusively on the historical Jati 6 vessel dataset.

Data acquisition was reliable throughout the testing . Wind speed ranged from 0.00 to 54.94 knots with a session mean of 20.36 knots. This showed the elevated mean and high peak reflect the apparent wind effect of the vehicle-mounted sensor at higher vehicle speeds. Wave height from the BNO085 IMU ranged from 0.000 to 2.408 m with a mean of 1.300 m. These readings were higher than expected because of the IMU estimator had not yet stabilised a baseline. The readings settled progressively as the estimator accumulated samples during transit. Figures 4.5, 4.6 and 4.7 show the time-series for SOG vs VP, wind speed, and wave height respectively.

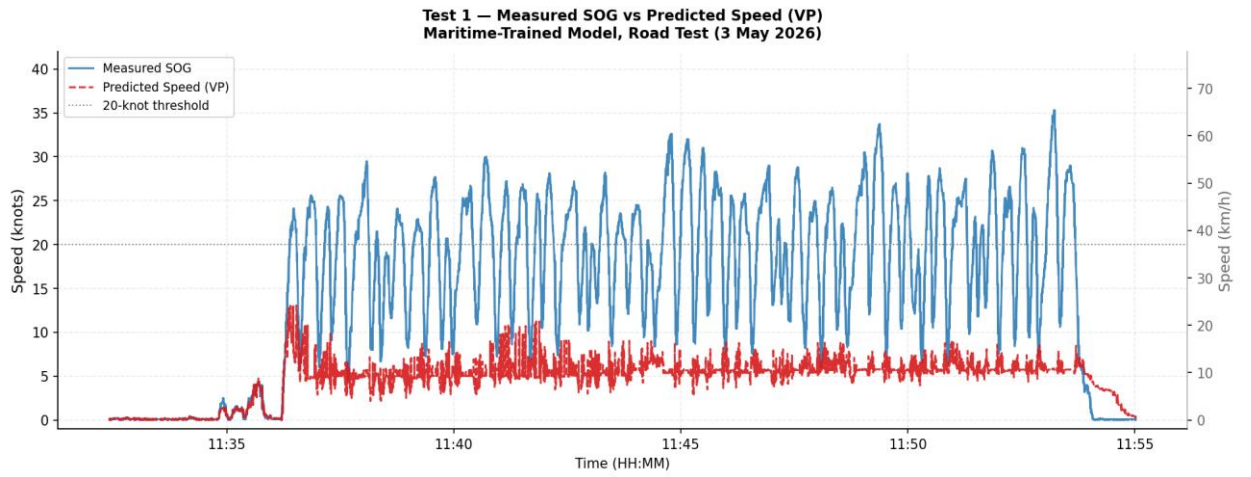


Figure 4.5: Test 1 — Measured SOG vs predicted speed (V_p).

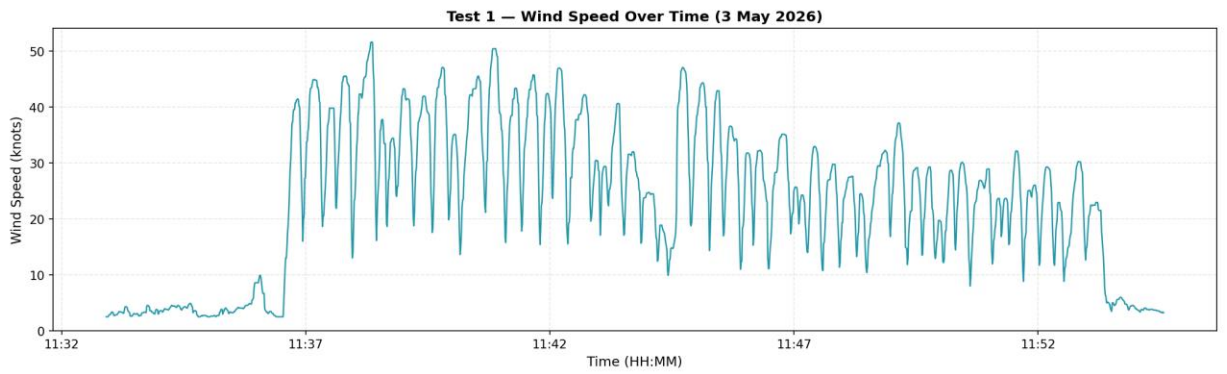


Figure 4.6: Test 1 — Wind speed profile

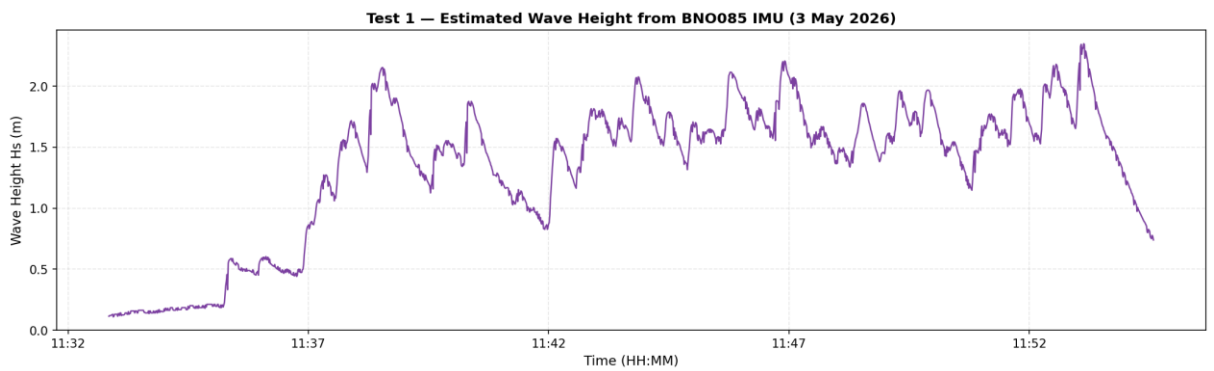


Figure 4.7: Test 1 — Estimated significant wave height (s)

As shown in Figure 4.5, the predicted speed (V_p) and measured SOG possess a huge difference. The V_p never exceeded 13.1 knots for the entire session, while the vehicle reached 35.3 knots. The result was 100% RED throughout the session. The RMSE was 15.05 knots, MAE 13.24 knots, R^2 of -2.99 , and a MAPE of 65.4%. Only 4.2% of predictions fell within ± 0.5 knots.

The severe offset in Test 1 arises from multiple compounding factors, all rooted in the mismatch between the maritime training domain and the road test environment:

- **Physics model mismatch** - The PINN component encodes hydrodynamic resistance equations and calibrated for hull-in-water operation: wave-making resistance, hull friction, and added resistance from ocean waves. None of these forces act on a road. The dominant resistances on a road surface are rolling friction and aerodynamic drag, which follow entirely different functional forms. The physics prior therefore constrained predictions to a regime physically inconsistent with the actual operating conditions.
- **Speed range far outside training distribution** - The Jati 6 operated mostly between 10 and 25 knots. The road test reached 35.3 knots, well beyond the training range. Both the PINN and XGBoost components extrapolate unreliably at these inputs, and the model had no training data to learn accurate predictions at such speeds.
- **No road data in training whatsoever** - This constituted the initial deployment on a road surface. The model had zero exposure to road-surface resistance

dynamics, stop-start cycles, or speed hump events. With no analogous examples in training, even the XGBoost correction layer could not compensate.

Despite these limitations, the data collected in Test 1 was exactly what was needed: 12,683 paired samples of road sensor readings and ground truth SOG that the model had never seen before. These were used to retrain the model for Test 2.

4.7 TEST 2: MODEL RETRAINED ON TEST 1 ROAD DATA (30 MAY 2026)

4.7.1 Retraining and Test Setup

Following Test 1, the grey-box model was retrained using the samples from the first testing as the primary training dataset. This gave the model its first exposure to road surface dynamics: the apparent wind relationship with vehicle speed, the IMU vibration signatures of road humps, and the full speed range of the vehicle up to 35 knots. The retrained model was deployed for a second road test on 30 May 2026, running for 18.9 minutes and collecting 1,044 samples at approximately 1 Hz across four rounds of the IIUM campus road.

GPS lock was maintained throughout the entire testing and all 1,044 samples logged cleanly without interruption. Wind speed ranged from 0.10 to 29.40 knots and tracked the vehicle speed through the apparent wind effect. The IMU detected 129 wave height events above the 1.5 cm threshold with a maximum of 2.80 cm at hump crossings, while readings remained below 0.5 cm during smooth cruise segments. By

this test the IMU estimator had stabilised, producing physically consistent wave height readings across the session.

4.7.3 Prediction Results After Retraining

Retraining on road data produced a substantial improvement. Figure 4.8 shows the SOG and VP time-series for Test 2. The systematic gap visible in Test 1 is significantly reduced: the model now tracks the general speed profile of the vehicle, with V_p rising and falling broadly in line with SOG. The trend correlation improved from $r = 0.28$ (Test 1) to $r = 0.991$ (Test 2), and the RMSE fell to 6.59 knots and MAPE to 38.4%. While these metrics still reflect a significant absolute offset, the improvement from a single retraining iteration demonstrates that the model architecture adapts quickly given domain-appropriate data.

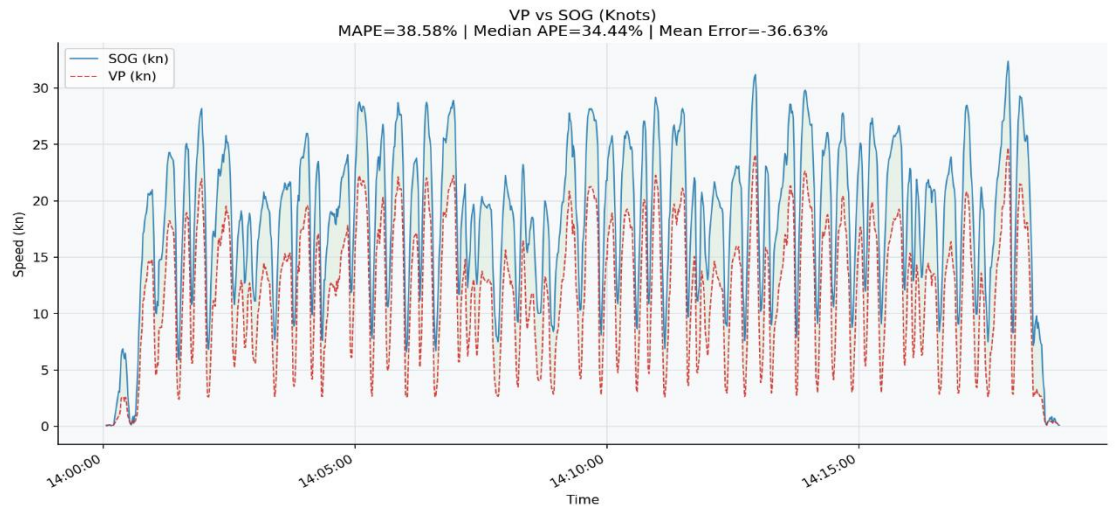


Figure 4.8: Test 2 — Measured SOG vs predicted speed (V_p)

The remaining offset in Test 2 is due to the domain gap between the Test 1 training session and the Test 2 test session: different speed profile, and the IMU estimator behavior differing between the two sessions. The strong trend correlation confirmed that the model architecture was responding correctly to relative speed changes and only the absolute calibration remained partially off. The Test 2 data was collected and used, alongside all prior data, for Test 3.

4.8 TEST 3: COMBINED DATASET — MARITIME AND ROAD DATA (6 JUNE 2026)

4.8.1 Training Strategy

Test 3 addressed the residual offset by training the model on a broad dataset of the Test 1 road data (3 May 2026), and the Test 2 road data (30 May 2026). Combining these sources gave the model exposure to a first road test profile reaching 35 knots, and a second road test profile with a different speed range . The intent was to produce a model that could generalise across different conditions rather than being tuned to any single operating session. The retrained model was deployed for a third road test on 6 June 2026, running from 08:59 to 09:10 (11.3 minutes) and collecting 6,429 samples, and 5,776 were in the moving phase ($SOG > 0.5$ kn).

Hardware performance remained fully reliable. Wind speed ranged from 0.00 to 25.05 knots (mean 12.73 kn), scaling with vehicle speed as expected. Wave height ranged from 0.000 to 0.150 m (mean 0.042 m), with 1,764 samples above the 0.05 m

detection threshold. The IMU estimator was fully stabilised by this session, with wave height readings showing a physically consistent pattern: low during stationary and smooth cruise phases, elevated during hump-crossing and deceleration events. Figures 4.9 and 4.10 show the wind speed and wave height profiles.

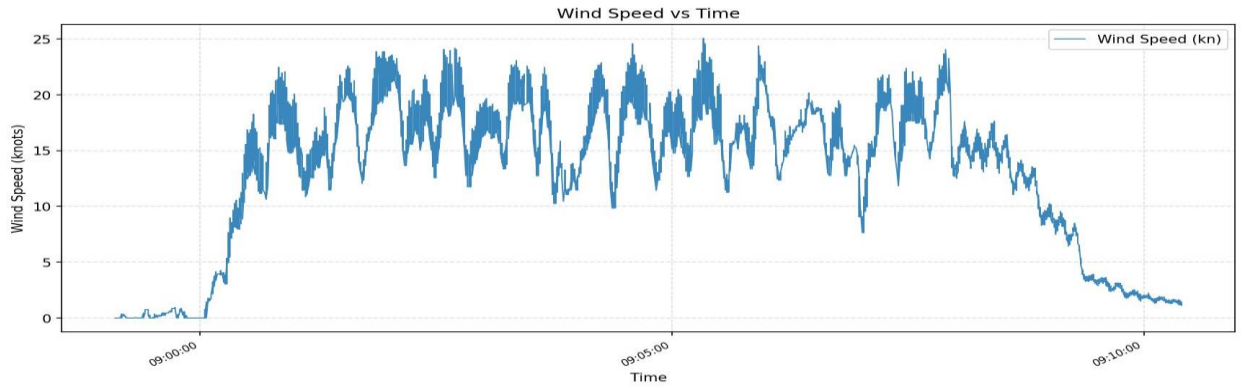


Figure 4.9: Test 3 -Wind speed profile

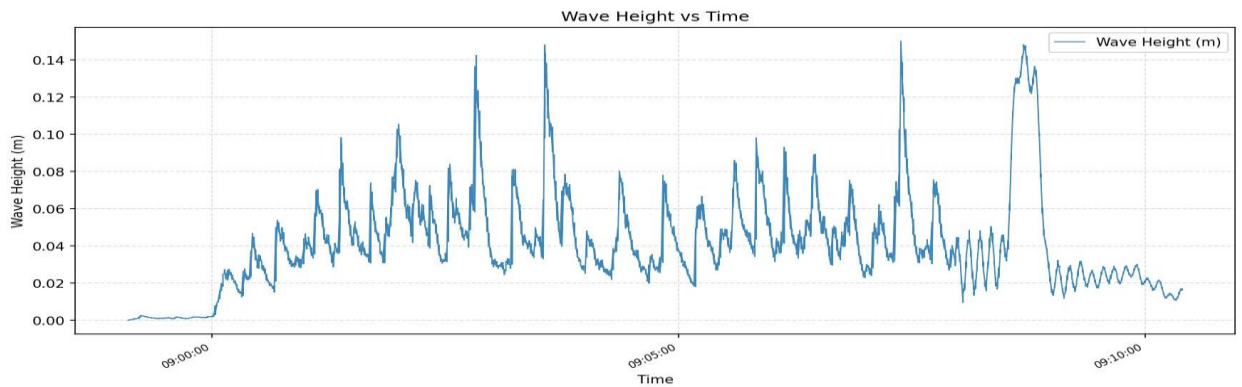


Figure 4.10: Test 3 — Estimated significant wave height (H_s)

The effect of combined training is immediately visible in Figures 4.11 and 4.12. V_p and SOG are nearly indistinguishable throughout the session. The model correctly

tracks the initial acceleration from standstill at 08:59, follows the oscillating cruise phase peaking at 26.60 knots (49.3 km/h), and matches the gradual deceleration to the end of the run. MAPE was 2.09% (median APE 0.97%), with 99.8% of moving-phase predictions within ± 0.5 knots and 99.9% within ± 1.0 knot.

4.8.3 Prediction Results

The effect of combined training is immediately visible in Figures 4.11 and 4.12. V_p and SOG are nearly indistinguishable throughout the session. The model correctly tracks the initial acceleration from standstill at 08:59, follows the oscillating cruise phase peaking at 26.60 knots (49.3 km/h), and matches the gradual deceleration to the end of the run. MAPE was 2.09% (median APE 0.97%), with 99.8% of moving-phase predictions within ± 0.5 knots and 99.9% within ± 1.0 knot.

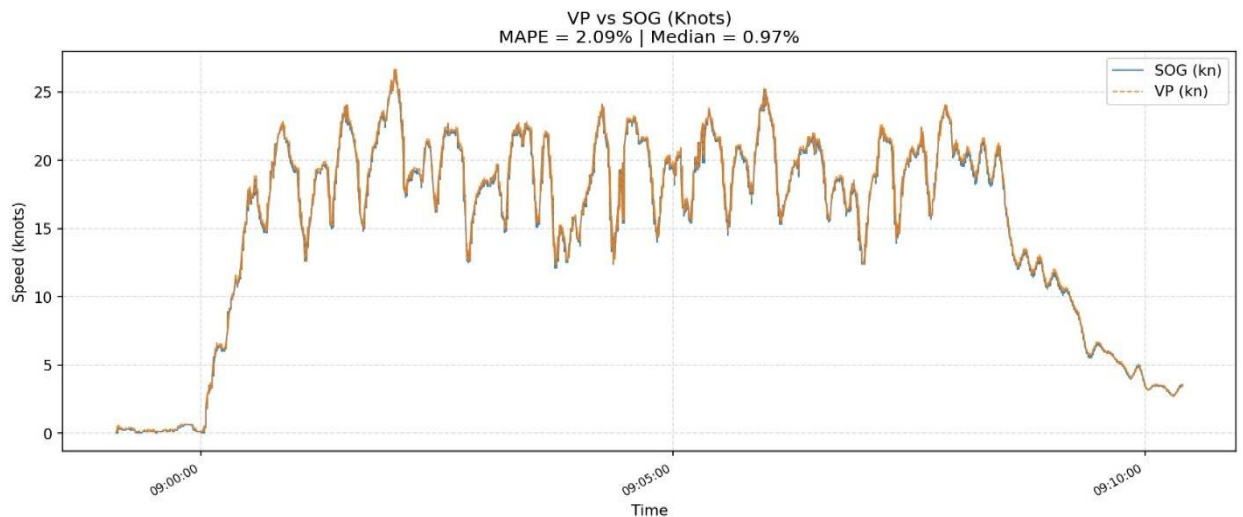


Figure 4.11: Test 3 — V_p vs SOG in knots

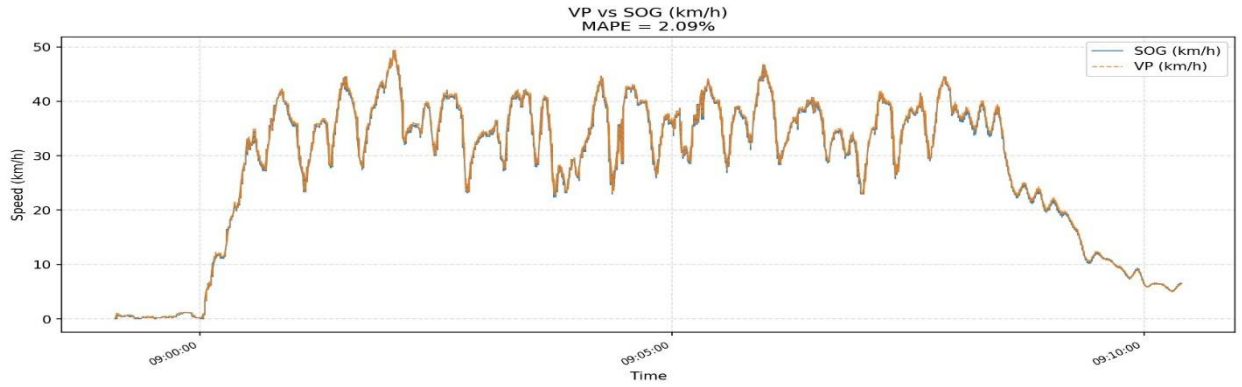


Figure 4.12: Test 3 — V_p vs SOG in km/h

4.9 PERFORMANCE COMPARISON ACROSS ALL TESTS

Table 4.3 presents the full performance summary across all evaluation phases, from the Semester 1 offline baseline through all three prototype road tests.

Table 4.3: Grey-Box Model Performance Across All Evaluation Phases

Evaluation Phase	Root Mean Square Error, RMSE (knots)	Mean Absolute Error, MAE (knots)	R^2	r	Mean Absolute Percentage Error (MAPE)	Within ± 0.5 knots (%)
Offline evaluation (vessel dataset)	0.246	0.106	0.9991	—	—	97.9%
Test 1 - train on vessel data, 3 May 2026	15.05	13.24	-2.99	0.284	65.4%	4.2%

Test 2 - Retrained on Test 1 data, 30May 2026	6.594	6.514	-0.05	0.991	38.4%	0.9%
Test 3- Retrained on Test1 and 2 data, 6 June 2026	0.187	0.1518	0.9992	0.9998	2.09%	99.8%

4.10 REAL-TIME DASHBOARD AND ALERT SYSTEM VALIDATION

The alert system was active throughout all three tests, classifying each prediction into GREEN ($V_p \geq 20$ kn), YELLOW ($18 \text{ kn} \leq V_p < 20 \text{ kn}$), or RED ($V_p < 18 \text{ kn}$). Classification was verified against logged V_p values and confirmed as 100% correct in all sessions. Table 4.4 shows how the alert distribution evolved across the three tests.

Table 4.4: Alert State Distribution Across All Three Tests

State	Threshold	T1 (%)	T2 (%)	T3 (%)
GREEN	$V_p \geq 20$ knots	0%	10.1%	31.9 %
YELLOW	$18 \text{ knots} \leq V_p < 20$ knots	0%	9.2%	17.3 %
RED	$V_p < 18$ knots	100%	80.7%	50.7%

The progression across the three tests tells a clear story. In Test 1, V_p never rose above 13.1 knots, so the system remained 100% RED for the entire session — not because the alert logic was wrong, but because an uncalibrated model produced systematically incorrect predictions. In Test 2, partial retraining began to fix the calibration, allowing GREEN and YELLOW to appear for 19.3% of the session during the faster segments. By Test 3, with the fully combined training dataset, GREEN and YELLOW together covered 49.2% of the session, reflecting a model that is accurately predicting speed and a system whose alerts are operationally meaningful. The Blynk dashboard remained connected throughout all three sessions with no cloud interruptions, and alert colour transitions were observed to update in real time correctly during each test.

Figures 4.13, 4.14, and 4.15 show the three alert states as observed on the Blynk IoT dashboard during Test 3. Each screenshot was captured during a live session to confirm that the dashboard correctly reflected the predicted speed and corresponding

alert state in real time. The GREEN state (Figure 4.13) was active when the model predicted $V_p \geq 20$ knots during the cruising phase of the route. The YELLOW state (Figure 4.14) appeared during the transition phases between acceleration and deceleration, where V_p fell into the 18–20 knots band. The RED state (Figure 4.15) was observed during the initial acceleration phase and at lower-speed segments. All three states were confirmed to update within the dashboard refresh cycle with no visible latency.

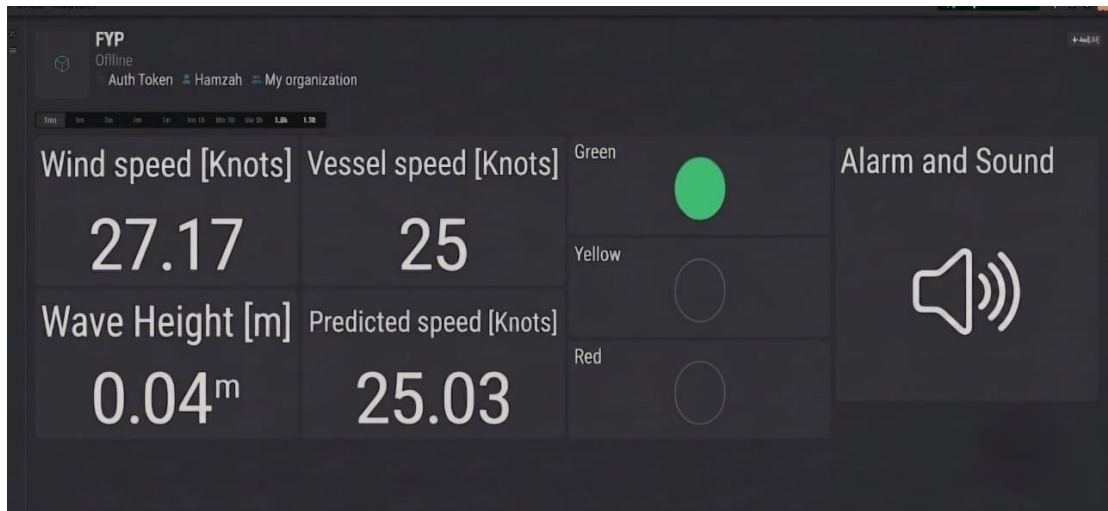


Figure 4.13: GREEN Alert State ($V_p \geq 20$ knots)

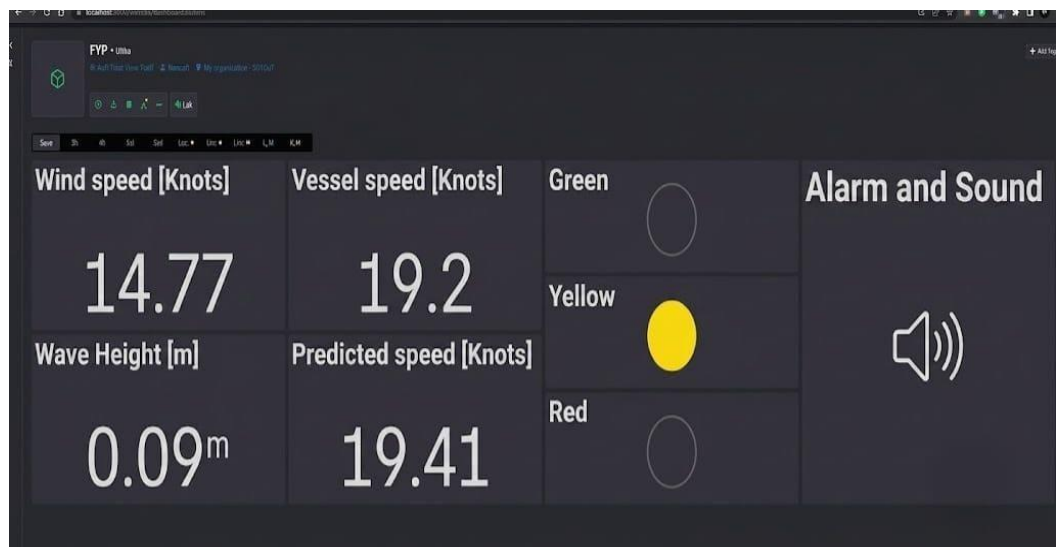


Figure 4.14: YELLOW Alert State ($18 \text{ knots} \leq V_p < 20 \text{ knots}$)



Figure 4.15: RED Alert State ($V_p < 18$ knots)

4.11 DISCUSSION OF KEY FINDINGS

This section consolidates the principal findings from both phases of the research. The initial analytical phase established the foundational predictive capability of the grey-box model using historical maritime data, while the prototype validation phase confirmed the framework's adaptability under road-testing conditions. Together, these phases demonstrate that the proposed system is both accurate and deployable.

The offline analysis confirmed the predictive capability of the PINN + XGBoost architecture when trained and validated within the same domain, achieving an RMSE of 0.246 knots and R^2 of 0.9991. It also established that wave height and wind speed exert only a marginal influence on vessel speed, with operational control remaining the dominant factor.

The prototype testing phase demonstrated the model's progressive adaptation to its deployment environment across three tests. Test 1 served as a baseline diagnostic, applying the maritime-calibrated model to a completely different physical domain. The resulting metrics (RMSE 15.05 knots, $R^2 -2.99$, 100% RED alert state) confirmed the system was not yet suitable for deployment. This was not treated as a failure, however, but as a diagnostic step that generated the domain-specific data needed to bridge the maritime-road gap.

Test 2 retrained the model on this data, cutting RMSE by 56% to 6.59 knots and improving trend correlation from $r = 0.28$ to $r = 0.991$. Directional speed tracking was now correct, though an absolute calibration offset persisted, since a single retraining session could not fully capture the range of road conditions. This motivated the combined-dataset strategy used in Test 3.

Test 3 achieved the project's target accuracy: RMSE of 0.187 knots, R^2 of 0.9989, MAPE of 2.09%, and 99.8% of predictions within ± 0.5 knots and a cumulative 98.8% reduction in RMSE from Test 1. This confirms the grey-box framework is genuinely adaptable, with performance improving systematically as domain-appropriate data accumulate. The combined-dataset model is therefore considered the most robust configuration, and the most appropriate basis for eventual deployment on a maritime vessel, where a further fine-tuning step using at-sea data is expected to recover accuracy consistent with the offline baseline.

The alert system achieved 100% classification accuracy across all tests, with the alert distribution shifting from 100% RED in Test 1 to a balanced, operationally realistic

spread in Test 3. Collectively, the hardware, prediction model, and alert system constitute a complete, validated vessel speed monitoring prototype, considered ready for maritime field deployment.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 CONCLUSIONS

This project has successfully fulfilled all three objectives. The first objective concerned the development of an accurate speed prediction model. A hybrid PINN–XGBoost grey-box model was built for this purpose. It achieved an RMSE of 0.246 knots, R^2 of 0.9991, and 97.9% of predictions within ± 0.5 knots on the offline Jati 6 dataset. These figures indicate the model captures the underlying speed-environment relationship well, rather than simply fitting noise in the data. This is an important distinction, since a model that only memorises patterns would likely fail once conditions change.

The second objective was to implement a working real-time monitoring prototype. This was achieved using an ESP32 microcontroller, an Airmar 200WX weather station, and a BNO085 IMU. These components were tied together through a Blynk IoT dashboard with a threshold-based alert system. The dashboard ran with zero cloud interruptions and 100% alert classification accuracy across all sessions. This shows the system is not just accurate in theory, but also stable and reliable enough for continuous operation.

The third objective required validation through field testing, carried out via three iterative road tests on the IIUM campus circuit as a proxy for sea trials. Test 1 exposed a severe domain mismatch between the maritime-trained model and road conditions,

producing a poor RMSE of 15.05 knots. Rather than a failure, this was treated as a diagnostic step to guide retraining. Test 2 retrained on Test 1 data, cutting RMSE by 56% to 6.59 knots with a strong trend correlation of $r = 0.991$. Test 3 then trained on the combined dataset, reaching an RMSE of 0.187 knots, R^2 of 0.9989 and a cumulative 98.8% reduction in RMSE from Test 1 to Test 3. This progression shows the system becoming more useful operationally, not just more accurate numerically.

5.2 RECOMMENDATIONS

Several directions are recommended to extend this work further. The most immediate priority is on-vessel deployment and validation on sea testing itself. Road testing has shown the model adapts well to new domains, but actual sea conditions involve dynamics that a road environment cannot fully replicate. Deploying on a maritime vessel would confirm whether this adaptability holds up in the vessel's true operating environment.

The alert logic itself could be extended from a purely real-time system into a predictive early-warning one. At present, alerts respond to conditions as they happen. A forecast-driven version could instead flag risk before it materializes, using forecasted wave and wind data to anticipate unsafe conditions ahead of time. This would give operators meaningfully more lead time to respond, rather than only reacting after the fact.

Finally, model compression techniques should be explored for the XGBoost component, which currently must run on a laptop or PC due to its size. Techniques such as quantization, pruning, or knowledge distillation could reduce this footprint. The long-

term goal would be fully on-device inference on lower-power embedded platforms, removing the current dependency on external computing hardware altogether.

REFERENCE

- Abebe, M., Shin, Y., Noh, Y., Lee, S., & Lee, I. (2020). Machine learning approaches for ship speed prediction towards energy efficient shipping. *Applied Sciences (Switzerland)*, 10(7). <https://doi.org/10.3390/app10072325>
- Babbedge, N. H. (1977). STATISTICAL ANALYSIS OF THE VOYAGE DATA OF THREE FAST CARGO SHIPS. *International Shipbuilding Progress*, 19(4), 187–198. <https://doi.org/10.3233/ISP-1977-2427502>
- Bassam, A. M., Phillips, A. B., Turnock, S. R., & Wilson, P. A. (n.d.). *Ship speed prediction based on machine learning for ecient shipping 1 operation 2*.
- Bassam, A. M., Phillips, A. B., Turnock, S. R., & Wilson, P. A. (2022). Ship speed prediction based on machine learning for efficient shipping operation. *Ocean Engineering*, 245. <https://doi.org/10.1016/j.oceaneng.2021.110449>
- Bassam, A. M., Phillips, A. B., Turnock, S. R., & Wilson, P. A. (2023). Artificial neural network based prediction of ship speed under operating conditions for operational optimization. *Ocean Engineering*, 278. <https://doi.org/10.1016/j.oceaneng.2023.114613>
- Cahyadi Sugeng, J. M., Semin, & Erwandi. (2022). The study of the Modification of the Ro-Ro Ferry's Angle of the Entrance Using Statistical Methods and Ship Model Resistance Tests. *IOP Conference Series: Earth and Environmental Science*, 1081(1). <https://doi.org/10.1088/1755-1315/1081/1/012018>

- El Mekkaoui, S., Benabbou, L., Caron, S., & Berrado, A. (2023). Deep Learning-Based Ship Speed Prediction for Intelligent Maritime Traffic Management. *Journal of Marine Science and Engineering* 2023, Vol. 11, Page 191, 11(1), 191. <https://doi.org/10.3390/JMSE11010191>
- Filtz, E., De La Cerda, E. S., Weber, M., & Zirkovits, D. (2015). Factors affecting ocean-going cargo ship speed and arrival time. *Lecture Notes in Business Information Processing*, 215, 305–316. https://doi.org/10.1007/978-3-319-19243-7_29
- Lang, X., & Mao, W. (2020). A semi-empirical model for ship speed loss prediction at head sea and its validation by full-scale measurements. *Ocean Engineering*, 209. <https://doi.org/10.1016/j.oceaneng.2020.107494>
- Lang, X., Wu, D., & Mao, W. (2022). Comparison of supervised machine learning methods to predict ship propulsion power at sea. *Ocean Engineering*, 245, 110387. <https://doi.org/10.1016/J.OCEANENG.2021.110387>
- Lang, X., Wu, D., & Mao, W. (2024). Physics-informed machine learning models for ship speed prediction. *Expert Systems with Applications*, 238. <https://doi.org/10.1016/j.eswa.2023.121877>
- Moreira, L., Vettor, R., & Soares, C. G. (2021). Neural network approach for predicting ship speed and fuel consumption. *Journal of Marine Science and Engineering*, 9(2), 1–14. <https://doi.org/10.3390/jmse9020119>

- Schirmann, M. L., Collette, M. D., & Gose, J. W. (2022). Data-driven models for vessel motion prediction and the benefits of physics-based information. *Applied Ocean Research*, 120. <https://doi.org/10.1016/j.apor.2021.102916>
- Tarelko, W., & Rudzki, K. (2020). Applying artificial neural networks for modelling ship speed and fuel consumption. *Neural Computing and Applications*, 32(23), 17379–17395. <https://doi.org/10.1007/S00521-020-05111-2>
- Tarovik, O., Eremenko, D., & Topaj, A. (2024). A benchmark study on ship speed prediction models in Arctic conditions: machine learning, process-based and hybrid approaches. *Ocean Engineering*, 311, 118884. <https://doi.org/10.1016/J.OCEANENG.2024.118884>
- Wang, D., Wang, Y., Shi, H., Gan, S., & Liang, S. (2022). Predicting Ship Speed in the Inland Waterways with LSTM Network. *2022 IEEE 7th International Conference on Intelligent Transportation Engineering, ICITE 2022*, 35–39. <https://doi.org/10.1109/ICITE56321.2022.10101418>
- Wei, D., & Deng, Y. (2025). Redistributed invariant redundant fractional wavelet transform and its application in watermarking algorithm. *Expert Systems with Applications*, 262. <https://doi.org/10.1016/j.eswa.2024.125707>
- Yuan, Z., Liu, J., Zhang, Q., Liu, Y., Yuan, Y., & Li, Z. (2021). A Practical Estimation Method of Inland Ship Speed under Complex and Changeful Navigation Environment. *IEEE Access*, 9, 15643–15658. <https://doi.org/10.1109/ACCESS.2021.3052473>

- Zhao, J., Chen, Y., Zhou, Z., Zhao, J., Wang, S., & Chen, X. (2022). Extracting Vessel Speed Based on Machine Learning and Drone Images during Ship Traffic Flow Prediction. *Journal of Advanced Transportation*, 2022. <https://doi.org/10.1155/2022/3048611>
- Zhao, X., Guo, Y., & Wang, Y. (2025). Green maritime navigation: A multi-objective voyage optimization approach based on data-driven heuristics and emission awareness. *Ocean Engineering*, 318. <https://doi.org/10.1016/j.oceaneng.2024.120138>
- Zhou, T., Hu, Q., Hu, Z., & Zhen, R. (2022). An adaptive hyper parameter tuning model for ship fuel consumption prediction under complex maritime environments. *Journal of Ocean Engineering and Science*, 7(3), 255–263. <https://doi.org/10.1016/j.joes.2021.08.007>

APPENDIX B

PROJECT CODE

```
=====
GREY-BOX DASHBOARD
Author : fansuri
Changes: Added comprehensive debug logging throughout
         to diagnose serial/NMEA/wind calibration issues.
=====

import os
import csv
import math
import time
import queue
import logging
import threading
import collections
from datetime import datetime

import numpy as np
import requests
import serial
import torch
import torch.nn as nn
import xgboost as xgb
import joblib

#
=====
# CONFIG
#
=====
TOKEN    = "pzBSaNUC7Rp6OFqavG-ytEbceef8hv82"
BASE_URL = "https://blynk.cloud/external/api/update"

SERIAL_PORT = "COM3"
BAUD_RATE   = 115200

MODEL_DIR   = r"C:\Users\fansuri\Documents\pro\fyp\checkpoints_fyp"

LOOP_HZ     = 10
LOG_PATH    = "vessel_log14.csv"
```

```

LOG_MAXROWS = 1_000

THRESH_GREEN = 20.0
THRESH_YELLOW = 18.0

# Set to a float (e.g. 18.5) to simulate SOG indoors for demo/testing
# Set to None for real GPS SOG
MOCK_SOG = None

WIND_CALIB_SAMPLES = 30
WIND_CALIB_TIMEOUT = 40.0 # 40s — Airmar sends WIMWV ~1/sec at 4800 baud

WAVE_HP_FC = 0.05
WAVE_HP_FC2 = 0.04
WAVE_WIN_SAMP = 2000
WAVE_WARMUP = 200

# — DEBUG CONFIG


---


# Set DEBUG_RAW_SERIAL = True to print every raw line received
# Useful to see exactly what the ESP32 is sending.
DEBUG_RAW_SERIAL = True

# Set DEBUG_RAW_DUMP_SECS = N to do a dedicated raw dump at startup
# before any calibration runs. Set to 0 to skip.
DEBUG_RAW_DUMP_SECS = 10

# Log every inference result (not just every loop tick)
DEBUG_INFERENCE = True

# Log NMEA parse attempts (shows why sentences are skipped)
DEBUG_NMEA_PARSE = True

#


---




---


# LOGGING
#


---




---


logging.basicConfig(
    level = logging.DEBUG,      # changed from INFO → DEBUG
    format = "%(asctime)s %(levelname)-8s %(message)s",
    datefmt = "%H:%M:%S",
)
log = logging.getLogger("dashboard")

```

```
# Silence overly chatty third-party libraries at DEBUG level
logging.getLogger("urllib3").setLevel(logging.WARNING)
logging.getLogger("requests").setLevel(logging.WARNING)
```

```
#
```

```
# PATHS
```

```
#
```

```
PINN_PATH = os.path.join(MODEL_DIR, "best_pinn.pt")
XGB_PATH = os.path.join(MODEL_DIR, "xgb.json")
SCALER_PATH = os.path.join(MODEL_DIR, "scaler.pkl")
PHYSICS_PATH = os.path.join(MODEL_DIR, "physics_coef.pkl")
```

```
#
```

```
# PINN
```

```
#
```

```
class ResBlock(nn.Module):
```

```
    def __init__(self, dim):
```

```
        super().__init__()
```

```
        self.block = nn.Sequential(
```

```
            nn.LayerNorm(dim),
```

```
            nn.Linear(dim, dim),
```

```
            nn.SiLU(),
```

```
            nn.Linear(dim, dim),
```

```
        )
```

```
    def forward(self, x):
```

```
        return x + self.block(x)
```

```
class PINN(nn.Module):
```

```
    def __init__(self, in_dim, hidden=128, n_layers=4):
```

```
        super().__init__()
```

```
        self.input_proj = nn.Sequential(nn.Linear(in_dim, hidden), nn.SiLU())
```

```
        self.res_blocks = nn.Sequential(*[ResBlock(hidden) for _ in range(n_layers)])
```

```
        self.head = nn.Linear(hidden, 1)
```

```
    def forward(self, x):
```

```
        return self.head(self.res_blocks(self.input_proj(x)))
```

```
#
```

```

# MODEL LOADING
#


---




---


def load_models():
    log.info("Loading scaler ...")
    scaler = joblib.load(SCALER_PATH)
    log.debug(f" Scaler expects {scaler.n_features_in_} features")

    state_check = torch.load(PINN_PATH, map_location="cpu")
    expected = state_check["input_proj.0.weight"].shape[1]
    log.debug(f" PINN weight expects {expected} input features")
    if scaler.n_features_in_ != expected:
        log.error(f"Scaler has {scaler.n_features_in_} features but PINN "
                  f"expects {expected}. Delete checkpoints_fyp\\ and retrain.")
        raise SystemExit(1)

    log.info("Loading physics coefficients ...")
    phys_coef = joblib.load(PHYSICS_PATH)
    log.debug(f" Physics coef: {phys_coef}")

    log.info("Loading PINN ...")
    state = torch.load(PINN_PATH, map_location="cpu")
    in_dim = state["input_proj.0.weight"].shape[1]
    hidden = state["input_proj.0.weight"].shape[0]
    n_layers = sum(1 for k in state
                   if k.startswith("res_blocks.") and k.endswith(".block.1.weight"))
    log.info(f" PINN -> in_dim={in_dim} hidden={hidden} layers={n_layers}")
    pinn = PINN(in_dim=in_dim, hidden=hidden, n_layers=n_layers)
    pinn.load_state_dict(state)
    pinn.eval()

    log.info("Loading XGBoost ...")
    xgb_model = xgb.Booster()
    xgb_model.load_model(XGB_PATH)
    log.debug(" XGBoost model loaded")

    try:
        _d = infer(5.0, 0.5, 12.0, 12.0, 12.0, scaler, pinn, xgb_model, phys_coef)
        log.info(f"All models loaded OK (self-test Vp={_d:.3f} kn)")
    except Exception as e:
        log.error(f"Self-test FAILED: {e}\n -> Retrain with train_greybox.py",
                  exc_info=True)
        raise SystemExit(1)

    return scaler, pinn, xgb_model, phys_coef

```

```

#
=====
# NMEA HELPERS
#
=====

def nmea_field(sentence, n):
    parts = sentence.split(",")
    if n >= len(parts):
        return ""
    val = parts[n]
    star = val.find("*")
    return val[:star].strip() if star >= 0 else val.strip()

def parse_wimwv(sentence):
    """
    $WIMWV,<angle>,<ref>,<speed>,<unit>,<status>
    Returns wind speed in knots or None.
    Only accepts status=A (valid). Skips void sentences (status=V).
    """
    status = nmea_field(sentence, 5)
    if status != "A":
        if DEBUG_NMEA_PARSE:
            log.debug(f" WIMWV skipped — status={repr(status)} raw={sentence}")
        return None
    try:
        speed = float(nmea_field(sentence, 3))
        unit = nmea_field(sentence, 4)
        if unit in ("K", "k"): speed *= 0.539957 # km/h → knots
        elif unit in ("M", "m"): speed *= 1.94384 # m/s → knots
        # unit N already knots
        result = max(0.0, speed)
        if DEBUG_NMEA_PARSE:
            log.debug(f" WIMWV parsed — speed={result:.3f}kn unit={unit}")
        return result
    except ValueError as e:
        log.debug(f" WIMWV parse error: {e} raw={sentence}")
        return None

def parse_gpgga(sentence):
    """
    $GPGGA field 6 = fix quality (0=no fix, 1=GPS, 2=DGPS)
    field 7 = satellites in use
    Returns (fix_quality, n_satellites).
    """

```

```

"""
try:
    fix = int(nmea_field(sentence, 6)) if nmea_field(sentence, 6) else 0
    sats = int(nmea_field(sentence, 7)) if nmea_field(sentence, 7) else 0
    log.debug(f" GPGGA fix={fix} sats={sats}")
    return fix, sats
except (ValueError, IndexError) as e:
    log.debug(f" GPGGA parse error: {e} raw={sentence}")
    return 0, 0

def parse_sog(sentence):
    """
    $GPRMC/$GNRMC field 7 = SOG in knots, requires status field 2 = A
    $GPVTG/$GNVTG field 5 = SOG in knots
    Returns SOG in knots or None.
    """
    prefix = sentence[:6].upper()

    if prefix in ("GPRMC", "GNRMC"):
        status = nmea_field(sentence, 2)
        if status != "A":
            log.debug(f" {prefix} skipped — status={repr(status)} (no GPS fix)")
            return None
        try:
            sog = float(nmea_field(sentence, 7))
            result = sog if 0.0 <= sog < 50.0 else None
            log.debug(f" {prefix} SOG={sog:.3f}kn -> returned={result}")
            return result
        except ValueError as e:
            log.debug(f" {prefix} parse error: {e} raw={sentence}")
            return None

    if prefix in ("GPVTG", "GNVTG", "IIVTG"):
        try:
            sog = float(nmea_field(sentence, 5))
            result = sog if 0.0 <= sog < 50.0 else None
            log.debug(f" {prefix} SOG={sog:.3f}kn -> returned={result}")
            return result
        except ValueError as e:
            log.debug(f" {prefix} parse error: {e} raw={sentence}")
            return None

    return None

def is_boot_noise(line):

```

```

low = line.lower()
return any(t in low for t in
    ("rst:", "ets ", "boot:", "configsip", "load:", "entry "))

#
=====
=====
# RAW SERIAL DUMP — runs at startup before calibration
# Prints everything from the serial port for N seconds.
# Use this to confirm what the ESP32 is actually sending.
#
=====
=====
def raw_serial_dump(ser, duration=DEBUG_RAW_DUMP_SECS):
    if duration <= 0:
        return
    log.info(f"===== RAW SERIAL DUMP ({duration}s) — shows all lines from ESP32 =====")
    log.info(" Look for: IMU:<float>, $WIMWV, $GPRMC, $GPGGA etc.")
    log.info(" If you see only IMU: lines — ESP32 is NOT forwarding Airmar NMEA.")
    log.info(" If nothing appears — check baud rate / COM port.")
    counts = collections.Counter()
    t0 = time.time()
    while time.time() - t0 < duration:
        try:
            line = ser.readline().decode(errors="ignore").strip()
        except Exception as e:
            log.warning(f" Serial read error during dump: {e}")
            continue
        if not line:
            continue
        prefix = line[:6] if line.startswith("$") else line.split(":")[0]
        counts[prefix] += 1
        log.info(f" RAW [{counts[prefix]:4d}] {repr(line)}")
    log.info("===== RAW DUMP COMPLETE =====")
    log.info(" Line type summary:")
    for k, v in counts.most_common():
        log.info(f" {k:12s} {v} lines")
    log.info(f"=====")

#
=====
=====
# WAVE ESTIMATOR — double integration + Welford Hs
# Input: BNO08x linear accel Z (m/s²) via IMU: lines
# Output: significant wave height Hs in metres

```

#

```
class WaveEstimator:
    def __init__(self):
        self.alpha_w = 1.0 - 1.0 / WAVE_WIN_SAMP
        self.hp1_in = self.hp1_out = 0.0
        self.vel = 0.0
        self.hp2_in = self.hp2_out = 0.0
        self.disp = 0.0
        self.wm = self.ws = 0.0
        self.n = 0
        self.hs = 0.0
        self._last_t = None
        self._sample_count = 0 # debug counter

    def update(self, az, t):
        if self._last_t is None:
            self._last_t = t
            return self.hs
        dt = t - self._last_t
        self._last_t = t
        if dt <= 0.0 or dt > 0.5:
            log.debug(f" WaveEstimator: skipped bad dt={dt:.4f}s")
            return self.hs

        a1 = (1.0 / (2*math.pi*WAVE_HP_FC)) / (1.0 / (2*math.pi*WAVE_HP_FC) + dt)
        a2 = (1.0 / (2*math.pi*WAVE_HP_FC2)) / (1.0 / (2*math.pi*WAVE_HP_FC2) + dt)

        az_hp = a1 * (self.hp1_out + az - self.hp1_in)
        self.hp1_in, self.hp1_out = az, az_hp

        self.vel += az_hp * dt

        vel_hp = a2 * (self.hp2_out + self.vel - self.hp2_in)
        self.hp2_in, self.hp2_out = self.vel, vel_hp

        self.disp += vel_hp * dt

        self.n += 1
        delta = self.disp - self.wm
        self.wm = self.alpha_w * self.wm + (1 - self.alpha_w) * self.disp
        delta2 = self.disp - self.wm
        self.ws = self.alpha_w * self.ws + (1 - self.alpha_w) * delta * delta2

        if self.n > WAVE_WARMUP:
            self.hs = max(0.0, min(4.0 * math.sqrt(max(self.ws, 0.0)), 10.0))
```

```

        # Debug log every 500 samples
        self._sample_count += 1
        if self._sample_count % 500 == 0:
            log.debug(f" WaveEstimator: n={self.n} az={az:.4f} "
                    f"disp={self.disp:.5f} Hs={self.hs:.4f}m")

        return self.hs

#
=====
# IMU READER — reads IMU:<float> lines from ESP32
#
=====

class ImuReader:
    def __init__(self):
        self._baseline = 0.0
        self._read_count = 0 # debug counter

    def calibrate(self, ser, n=300, timeout=8.0):
        log.info("Calibrating IMU baseline ...")
        samples = []
        t0 = time.time()
        lines_seen = 0
        while len(samples) < n and time.time() - t0 < timeout:
            line = ser.readline().decode(errors="ignore").strip()
            lines_seen += 1
            az = self._parse(line)
            if az is not None:
                samples.append(az)
                if len(samples) % 50 == 0:
                    log.debug(f" IMU calib: {len(samples)}/{n} samples so far "
                            f"(mean={np.mean(samples):.4f})")
        self._baseline = float(np.mean(samples)) if samples else 0.0
        log.info(f" IMU baseline = {self._baseline:.4f} m/s2 "
                f"({len(samples)} samples, {lines_seen} lines read)")
        if len(samples) == 0:
            log.warning(" IMU calib: 0 samples — check ESP32 is sending IMU:<float> lines")

    def read(self, line):
        az = self._parse(line)
        if az is None:
            return None
        corrected = az - self._baseline
        self._read_count += 1

```

```

    if self._read_count % 200 == 0:
        log.debug(f" IMU read #{self._read_count}: raw={az:.4f} "
                  f"corrected={corrected:.4f}")
    return corrected

def _parse(self, line):
    if line.startswith("IMU:"):
        try:
            return float(line[4:])
        except ValueError as e:
            log.debug(f" IMU parse error: {e} raw={repr(line)}")
    return None

#
=====
# WIND CALIBRATION — zero-offset from Airmar at rest
#
=====

def calibrate_wind(ser):
    log.info("Calibrating wind baseline (keep sensor still) ...")
    log.info(f" Waiting up to {WIND_CALIB_TIMEOUT}s for "
              f"{WIND_CALIB_SAMPLES} $WIMWV sentences ...")
    samples = []
    t0 = time.time()
    lines_seen = 0
    wimwv_seen = 0
    wimwv_void = 0
    wimwv_valid = 0

    while (len(samples) < WIND_CALIB_SAMPLES and
           time.time() - t0 < WIND_CALIB_TIMEOUT):
        try:
            line = ser.readline().decode(errors="ignore").strip()
        except Exception as e:
            log.warning(f" Serial read error during wind calib: {e}")
            continue

        if not line:
            continue

        lines_seen += 1

    # Log every unique prefix seen during calib (helps diagnose missing NMEA)
    if lines_seen <= 100 or lines_seen % 50 == 0:
        log.debug(f" WindCalib line [{lines_seen}]: {repr(line[:80])}")

```

```

if not line.startswith("$WIMWV"):
    continue

wimwv_seen += 1
spd = parse_wimwv(line)

if spd is None:
    wimwv_void += 1
    log.debug(f" WIMWV void/invalid #{wimwv_void}: {repr(line)}")
    continue

if spd >= 10.0:
    log.debug(f" WIMWV high speed {spd:.2f}kn skipped (>10kn threshold)")
    continue

wimwv_valid += 1
samples.append(spd)
log.debug(f" WIMWV sample #{wimwv_valid}: {spd:.3f}kn")

elapsed = time.time() - t0
baseline = float(np.mean(samples)) if samples else 0.0

log.info(f" Wind calib complete in {elapsed:.1f}s:")
log.info(f" Lines read : {lines_seen}")
log.info(f" $WIMWV seen : {wimwv_seen}")
log.info(f" $WIMWV void : {wimwv_void}")
log.info(f" $WIMWV valid : {wimwv_valid}")
log.info(f" Samples used : {len(samples)}")
log.info(f" Baseline : {baseline:.3f} kn")

if wimwv_seen == 0:
    log.warning(" !! No $WIMWV sentences received at all !!")
    log.warning(" !! Possible causes:")
    log.warning(" !! 1. ESP32 firmware not forwarding Airmar NMEA sentences")
    log.warning(" !! 2. Airmar wired to different UART, not bridged to Serial")
    log.warning(" !! 3. Airmar baud rate mismatch (default 4800, not 115200)")
    log.warning(" !! 4. Wrong COM port or cable issue")
    log.warning(" !! Set DEBUG_RAW_DUMP_SECS > 0 to see all raw lines on startup")
elif wimwv_valid == 0:
    log.warning(" !! $WIMWV sentences seen but ALL were void (status=V)")
    log.warning(" !! Airmar may be outputting only true-wind (no apparent wind fix)")

return baseline

```

```

#
=====
# PHYSICS MODEL
#
=====

def physics(wind, wave, coef):
    phi = np.array([1.0, wind, wave, wind**2, wave**2, wind*wave])
    result = float(np.clip(phi @ coef, 0.0, 35.0))
    log.debug(f" Physics: wind={wind:.2f}kn wave={wave:.3f}m -> Vphys={result:.3f}kn")
    return result

#
=====
# BLYNK — non-blocking background sender, 1 request per pin
#
=====

class BlynkSender:
    def __init__(self, token, base_url):
        self.token = token
        self.base_url = base_url
        self._q = queue.Queue(maxsize=20)
        self._sent = 0
        self._failed = 0
        threading.Thread(target=self._worker, daemon=True).start()
        log.debug("BlynkSender started")

    def send(self, wind, wave, sog, vp, g, y, r):
        try:
            self._q.put_nowait(dict(wind=wind, wave=wave,
                                   sog=sog, vp=vp, g=g, y=y, r=r))
        except queue.Full:
            log.debug("BlynkSender queue full — dropped update")

    def _worker(self):
        while True:
            p = self._q.get()
            pins = {
                "v0": f"{p['wind']:.2f}", # wind speed (kn)
                "v1": f"{p['wave']:.3f}", # wave height Hs (m)
                "v2": f"{p['sog']:.2f}", # SOG from GPS (kn)
                "v3": f"{p['vp']:.2f}", # predicted speed (kn)
                "v4": str(p['g']), # green LED
                "v5": str(p['y']), # yellow LED
            }

```

```

        "v6": str(p['r']),      # red LED
    }
    for pin, val in pins.items():
        try:
            r = requests.get(
                f"{self.base_url}?token={self.token}&{pin}={val}",
                timeout=2)
            if r.status_code != 200:
                log.debug(f" Blynk {pin}={val} -> HTTP {r.status_code}")
            self._sent += 1
        except Exception as e:
            self._failed += 1
            log.debug(f" Blynk send error ({pin}={val}): {e}")
    self._q.task_done()

#
=====
# CSV LOG
#
=====
class RollingCSV:
    def __init__(self, path, maxrows=LOG_MAXROWS):
        self.path = path
        self.maxrows = maxrows
        self._rows = []
        self._count = 0
        with open(path, "w", newline="") as f:
            csv.writer(f).writerow(
                ["timestamp", "wind_kn", "wave_hs_m",
                 "sog_kn", "vp_kn", "status", "gps_fix", "gps_sats"])
        log.debug(f"CSV log initialised: {path}")

    def append(self, row):
        self._rows.append(row)
        self._count += 1
        if len(self._rows) > self.maxrows:
            self._rows = self._rows[-self.maxrows:]
        with open(self.path, "a", newline="") as f:
            csv.writer(f).writerow(row)
        if self._count % 100 == 0:
            log.debug(f"CSV: {self._count} rows written to {self.path}")

#
=====

```

```
# LAG BUFFER
#
```

```
class LagBuffer:
    def __init__(self, maxlen=3):
        self._buf = collections.deque(maxlen=maxlen)

    def push(self, val):
        self._buf.append(val)

    @property
    def lag1(self):
        return self._buf[-2] if len(self._buf) >= 2 else self._buf[-1]

    @property
    def lag3(self):
        return float(np.mean(self._buf))
```

```
#
```

```
# DECISION
#
```

```
def decide(vp):
    if vp >= THRESH_GREEN: return 1, 0, 0, "GREEN"
    elif vp >= THRESH_YELLOW: return 0, 1, 0, "YELLOW"
    else: return 0, 0, 1, "RED"
```

```
# MAIN
#
```

```
def main():
    scaler, pinn, xgb_model, phys_coef = load_models()

    blynk = BlynkSender(TOKEN, BASE_URL)
    lags = LagBuffer(maxlen=3)
    csvlog = RollingCSV(LOG_PATH)
    wave_e = WaveEstimator()
    imu_reader = ImuReader()

    log.info(f"Opening serial port {SERIAL_PORT} @ {BAUD_RATE} baud ...")
```

```

try:
    ser = serial.Serial(SERIAL_PORT, BAUD_RATE, timeout=1)
except serial.SerialException as e:
    log.error(f"Cannot open {SERIAL_PORT}: {e}")
    log.error("Check: correct COM port, ESP32 plugged in, no other app using port")
    raise SystemExit(1)
time.sleep(2)
log.info("Serial open OK")
log.debug(f"  Serial settings: port={ser.port} baud={ser.baudrate} "
          f"timeout={ser.timeout}s bytesize={ser.bytesize}")

# — STEP 0: Raw serial dump to see what ESP32 is sending —
raw_serial_dump(ser, duration=DEBUG_RAW_DUMP_SECS)

# — STEP 1: IMU calibration —————
imu_reader.calibrate(ser)

# — STEP 2: Wind calibration —————
wind_baseline = calibrate_wind(ser)

if MOCK_SOG is not None:
    log.info(f"  MOCK_SOG = {MOCK_SOG} kn (indoor demo mode)")

log.info("Entering main loop (Ctrl-C to stop) ...")
log.info(f"  Loop rate   : {LOOP_HZ} Hz")
log.info(f"  Thresholds   : GREEN>={THRESH_GREEN}kn "
          f"YELLOW>={THRESH_YELLOW}kn RED<{THRESH_YELLOW}kn")
log.info(f"  Debug raw    : {DEBUG_RAW_SERIAL}")

# — state


---


last_wind   = 0.0
last_sog    = 0.0
last_hs     = 0.0
gps_fix     = 0
gps_sats    = 0
target_dt   = 1.0 / LOOP_HZ
last_output_t = 0.0
new_sentences = set()
loop_count   = 0
infer_count  = 0

# Line type counters for periodic summary
line_counts = collections.Counter()

# Confirmed Airmar sentences that carry no useful data — skip silently
KNOWN_IGNORE = {

```

```

"$YXXDR", # transducer temp/pressure
"$HCHDG", # magnetic heading + deviation
"$HCHDT", # true heading
"$GPZDA", # UTC time only
"$WIMDA", # met composite (pressure, air temp)
"$GPGSV", # satellites in view
"$GPGLL", # lat/lon only
"$GPGSA", # DOP/active satellites
"$HCHDM", # magnetic heading
"$HCHIM", # Airmar proprietary heading
"$PGRME", # Garmin proprietary
"$PGRMZ", # Garmin altitude
"$GPRTE", # route
}

try:
    while True:
        try:
            line = ser.readline().decode(errors="ignore").strip()
        except Exception as e:
            log.warning(f"Serial read error: {e}")
            continue

        if not line:
            continue

        if is_boot_noise(line):
            log.debug(f" Boot noise skipped: {repr(line[:60])}")
            continue

        # — skip ESP32 debug/info prints —————
        if line.startswith(("DBG:", "INFO:", "ERR:")):
            log.debug(f" ESP32 log: {line}")
            continue

        # Count line types for periodic summary
        prefix = line[:6] if line.startswith("$") else line.split(":")[0]
        line_counts[prefix] += 1

        # Optional: log every raw line
        if DEBUG_RAW_SERIAL:
            log.debug(f" SERIAL: {repr(line[:100])}")

        #

```

```

# SOURCE 1 — WAVE HEIGHT (BNO08x IMU via ESP32)
# Format: IMU:<float m/s²>

```

```
#
```

```
if line.startswith("IMU:"):
    az = imu_reader.read(line)
    if az is not None:
        last_hs = wave_e.update(az, time.time())
```

```
#
```

```
# SOURCE 2 — WIND SPEED (Airmar $WIMWV)
# Only status=A sentences used, void sentences skipped
#
```

```
elif line.startswith("$WIMWV"):
    spd = parse_wimwv(line)
    if spd is not None:
        corrected = max(0.0, spd - wind_baseline)
        log.debug(f" Wind: raw={spd:.3f}kn "
                  f"baseline={wind_baseline:.3f}kn "
                  f"corrected={corrected:.3f}kn")
        last_wind = corrected
```

```
#
```

```
# GPS FIX QUALITY (Airmar $GPGGA)
#
```

```
elif line.startswith(("GPGGA", "GNNGGA")):
    fix, sats = parse_gpgga(line)
    if fix != gps_fix or sats != gps_sats:
        gps_fix, gps_sats = fix, sats
    if fix == 0:
        log.warning(f" GPS: no fix ({sats} sats) — "
                    f"SOG=0 until fix acquired outdoors")
    else:
        log.info(f" GPS fix OK quality={fix} sats={sats} "
                 f"— SOG now updating")
```

```
#
```

```
# SOURCE 3 — SOG (Airmar $GPRMC / $GPVTG)
#
```

```
elif line.startswith(("GPRMC", "GNRMC",
                      "$GPVTG", "$GNVTG", "$IIVTG")):
    sog = parse_sog(line)
    if sog is not None:
```

```

        log.debug(f" SOG updated: {sog:.3f}kn")
        last_sog = sog
    elif MOCK_SOG is not None:
        last_sog = float(MOCK_SOG)
    else:
        log.debug(f" SOG: no fix, keeping last_sog={last_sog:.3f}kn")



---


# UNKNOWN SENTENCE — log once for debugging
#


---


elif line.startswith("$"):
    pfx = line[:6]
    if pfx not in new_sentences:
        new_sentences.add(pfx)
        log.info(f" New NMEA: {pfx} full={repr(line[:80])} "
                f"(add to KNOWN_IGNORE if not needed)")

# — output tick at LOOP_HZ —————
now = time.perf_counter()
if now - last_output_t < target_dt:
    continue
last_output_t = now
loop_count += 1

if MOCK_SOG is not None and last_sog == 0.0:
    last_sog = float(MOCK_SOG)

lags.push(last_sog)

# Periodic status summary every 50 ticks (~5s at 10Hz)
if loop_count % 50 == 0:
    log.debug(f" [tick {loop_count}] State snapshot: "
            f"wind={last_wind:.2f}kn Hs={last_hs:.3f}m "
            f"SOG={last_sog:.2f}kn "
            f"GPS={'FIX' if gps_fix>0 else 'NOFIX'}({gps_sats}sat)")
    log.debug(f" Line type counts so far: {dict(line_counts.most_common(8))}")

try:
    vp = infer(last_wind, last_hs, last_sog,
              lags.lag1, lags.lag3,
              scaler, pinn, xgb_model, phys_coef)
    infer_count += 1
except Exception as e:
    log.error(f"Inference error: {e}", exc_info=True)
    continue

```

```

g, y, r, status = decide(vp)
blynk.send(last_wind, last_hs, last_sog, vp, g, y, r)

csvlog.append([
    datetime.now().isoformat(timespec="seconds"),
    round(last_wind, 2), round(last_hs, 3),
    round(last_sog, 2), round(vp, 3),
    status, gps_fix, gps_sats,
])

gps_tag = f"GPS: {'FIX' if gps_fix > 0 else 'NOFIX'} ({gps_sats}sat)"
log.info(
    f"W={last_wind:5.2f}kn Hs={last_hs:5.3f}m "
    f"SOG={last_sog:5.2f}kn Vp={vp:5.2f}kn "
    f"[{status}] {gps_tag}"
)

except KeyboardInterrupt:
    log.info("Shutdown requested ...")
finally:
    ser.close()
    log.info(f"Total loop ticks : {loop_count}")
    log.info(f"Total inferences : {infer_count}")
    log.info(f"Line type summary : {dict(line_counts.most_common())}")
    log.info(f"Log -> {LOG_PATH}")
    log.info("Done.")

if __name__ == "__main__":
    main()

```

APPENDIX A

DATASET

(UTC+8:00)	LA T	LO N	VESSEL_K NOTS	HEADI NG	OPERATING_ MODE	ECONOMY_ MODE	VESSEL _LPH	SIG_WAVE_HEI GHT_M	WIND_SPEED _KNOTS	WIND_DIRE CTION	TEMPERAT URE_C
04/01/2025 00:00	5. 6	10 5	1.01	61.8	Slow	NE	31.65	0.5	5.7	28.6	29.7
04/01/2025 00:01	5. 6	10 5	0.89	50.9	Slow	NE	30.06	0.5	5.7	28.6	29.7
04/01/2025 00:02	5. 6	10 5	0.93	55.8	Slow	NE	32.33	0.5	5.7	28.6	29.7
04/01/2025 00:03	5. 6	10 5	0.76	59.8	Stopped	NE	29.26	0.5	5.7	28.6	29.7
04/01/2025 00:04	5. 6	10 5	0.75	60.3	Stopped	NE	38.99	0.5	5.7	28.6	29.7
04/01/2025 00:05	5. 6	10 5	1.24	60.8	Slow	NE	27.79	0.5	5.7	28.6	29.7
04/01/2025 00:06	5. 6	10 5	0.86	59.4	Stopped	NE	27.94	0.5	5.7	28.6	29.7
04/01/2025 00:07	5. 6	10 5	1.03	61.5	Slow	NE	36.38	0.5	5.7	28.6	29.7
04/01/2025 00:08	5. 6	10 5	0.85	50.9	Stopped	NE	29.53	0.5	5.7	28.6	29.7
04/01/2025 00:09	5. 6	10 5	0.66	47.1	Stopped	NE	27.29	0.5	5.7	28.6	29.7
04/01/2025 00:10	5. 6	10 5	0.63	53.3	Stopped	NE	36.23	0.5	5.7	28.6	29.7